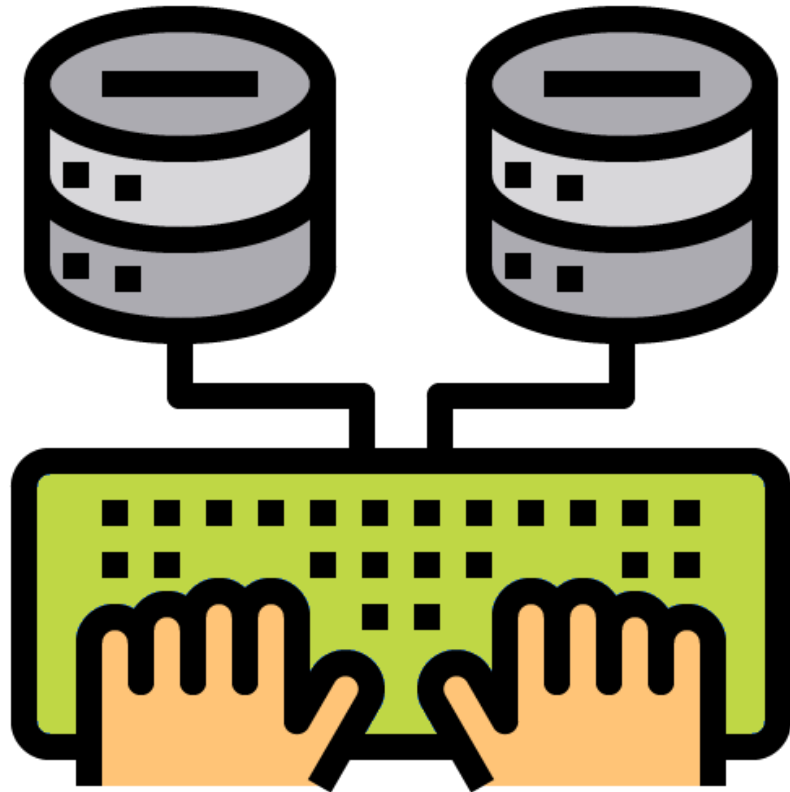




Big Data Analytics

Hadoop and Distributed FS

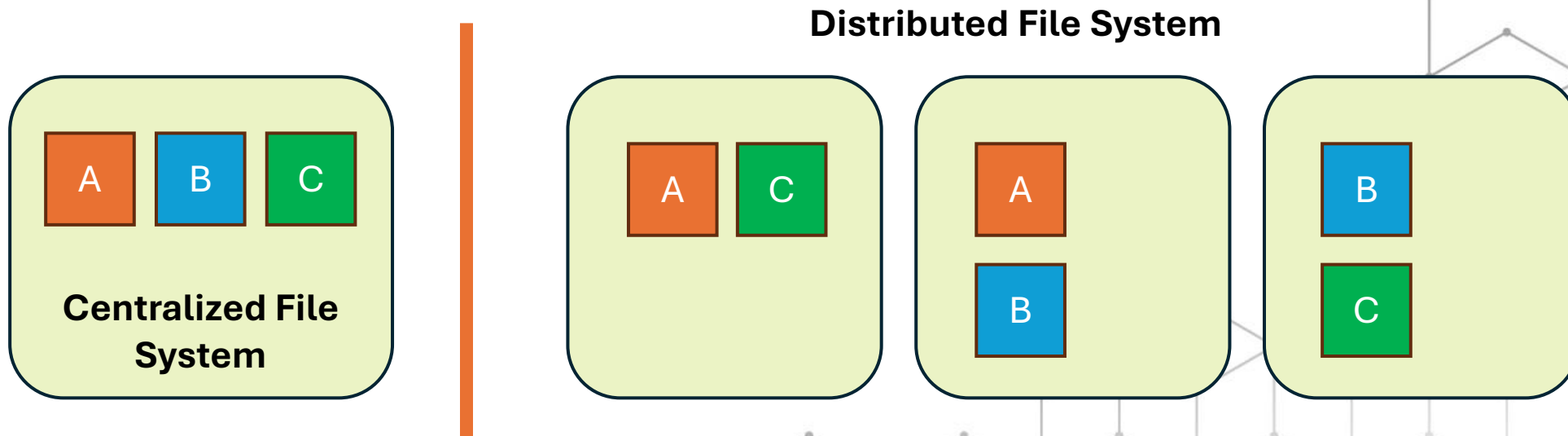
Lecturer:

Márcia Lourenço Baptista
m.baptista@novaims.unl.pt

Learning Objectives

By the end of this lecture, students should be able to:

- Understand the Fundamentals of Big Data Storage
- Examine Distributed Storage Systems
- Understand the role of Hadoop Distributed File System (HDFS) and its architecture



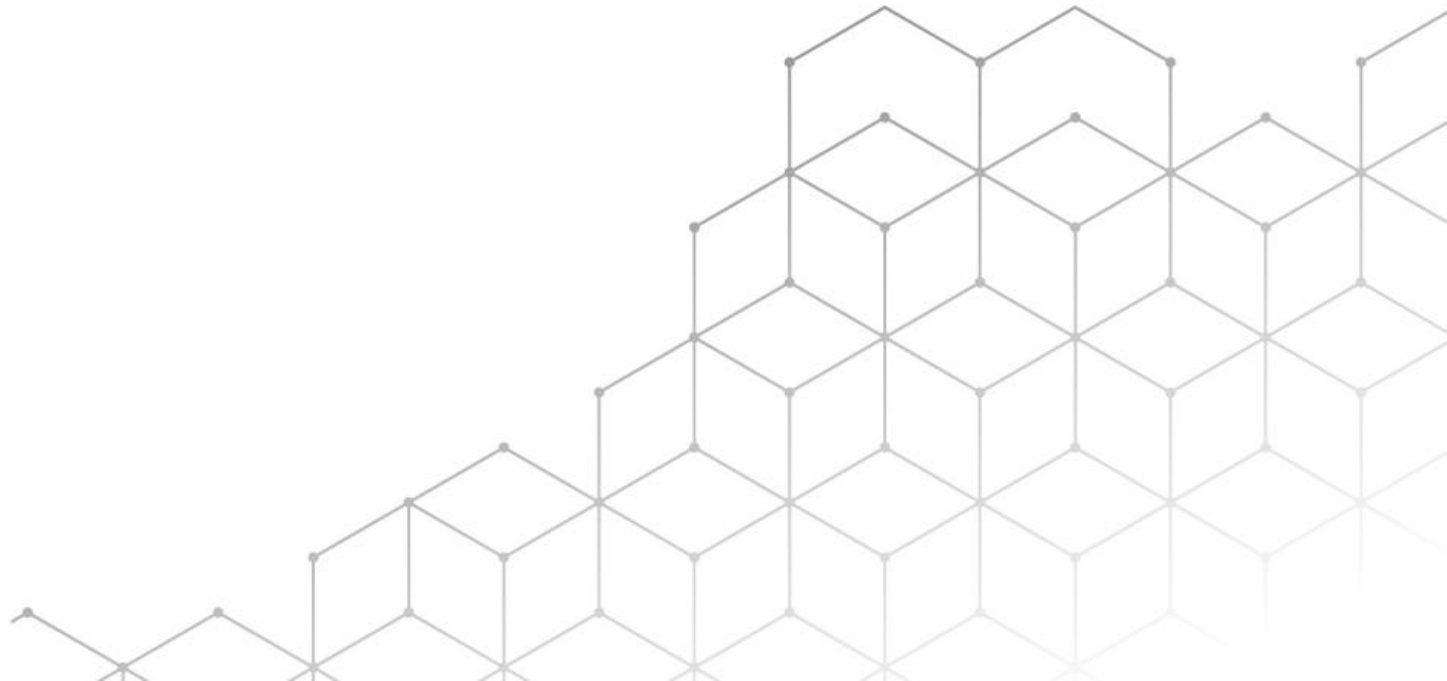
Agenda for Today...

- Types of Data
 - Structured Data
 - Semistructured Data
 - Unstructured Data
- Distributed File System (DFS)
- How DFS Works?
- Operations on DFS
- Fault Tolerance by Replication
- Concurrency and Multi-User Access
- Hadoop Distributed File System (HDFS)
- Databricks File System (DBFS)



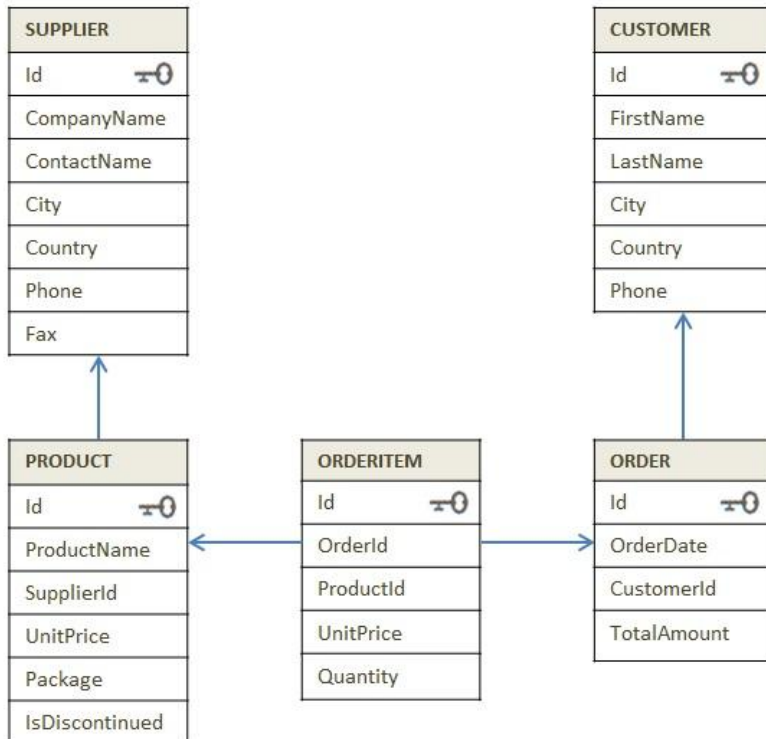
Questions

- Is Excel Spreadsheet Structured Data?
- What is Hadoop Distributed File System (HDFS)?
- What are the differences between block storage, object storage, and file storage?



Types of Data

- Data is generally categorized into three types: **structured**, **semi-structured**, and **unstructured data**. The key differences among them are based on their format and ease of processing.



```
{
  "sample.json" > [ ] data
  1 {
  2   "data": [
  3     {
  4       "type": "articles",
  5       "id": "1",
  6       "attributes": {
  7         "title": "Working with JSON Data in python",
  8         "description": "This article explains the various ways to work with JSON data in python.",
  9         "created": "2020-12-28T14:56:29.000Z",
 10        "updated": "2020-12-28T14:56:28.000Z"
 11       },
 12       "author": {
 13         "id": "1",
 14         "name": "Aveek Das"
 15       }
 16     }
 17   ]
 18 }
```



Concrete Examples

Unstructured data

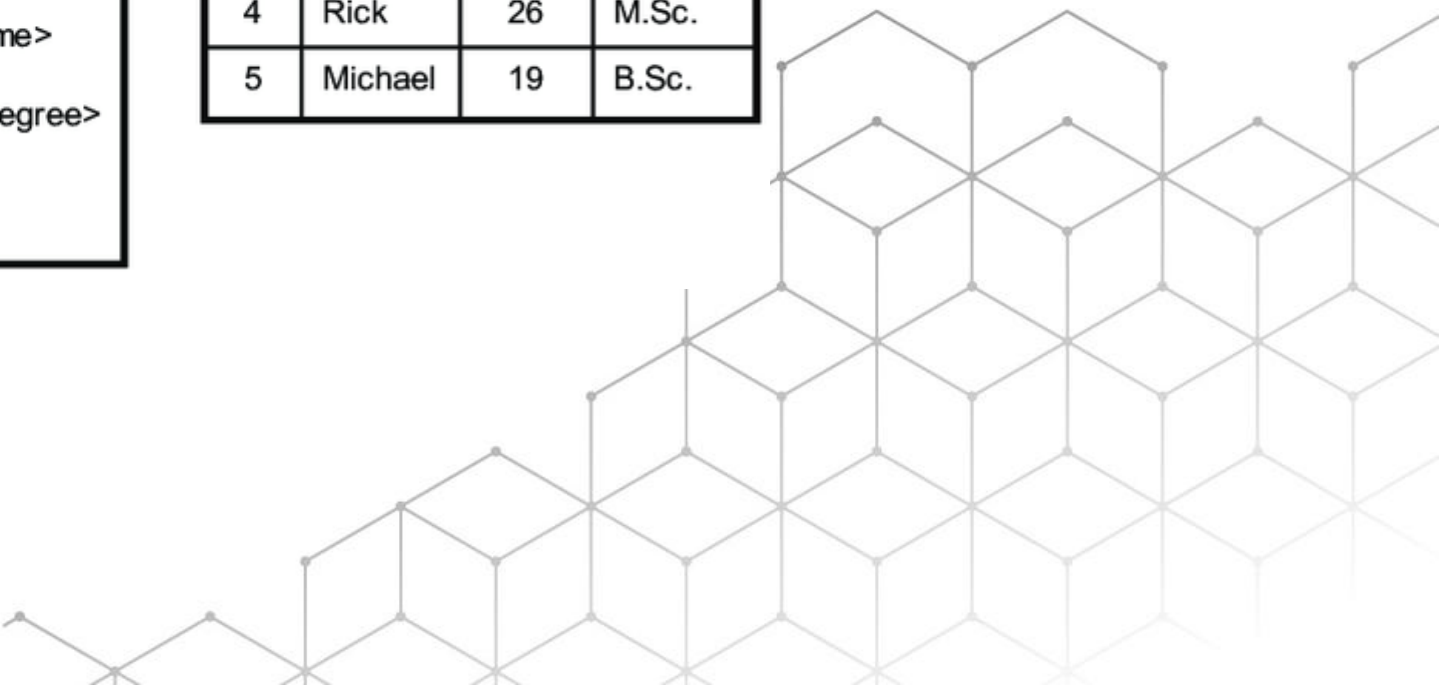
The university has 5600 students.
John's ID is number 1, he is 18 years old and already holds a B.Sc. degree.
David's ID is number 2, he is 31 years old and holds a Ph.D. degree. Robert's ID is number 3, he is 51 years old and also holds the same degree as David, a Ph.D. degree.

Semi-structured data

```
<University>
  <Student ID="1">
    <Name>John</Name>
    <Age>18</Age>
    <Degree>B.Sc.</Degree>
  </Student>
  <Student ID="2">
    <Name>David</Name>
    <Age>31</Age>
    <Degree>Ph.D. </Degree>
  </Student>
  ....
</University>
```

Structured data

ID	Name	Age	Degree
1	John	18	B.Sc.
2	David	31	Ph.D.
3	Robert	51	Ph.D.
4	Rick	26	M.Sc.
5	Michael	19	B.Sc.



Structured Data

- Data is represented in a tabular format (columns and rows)
- All records are field-consistent, and fields are well-defined.
- Can be stored on **Relational Databases**
- Algorithmically friendly (easy to search and filter)
- Information is dense and high quality
- **Examples:** Relation Databases (SQL)
- **Question:** Is Excel Spreadsheet Structured Data?



Where Excel is Structured Data

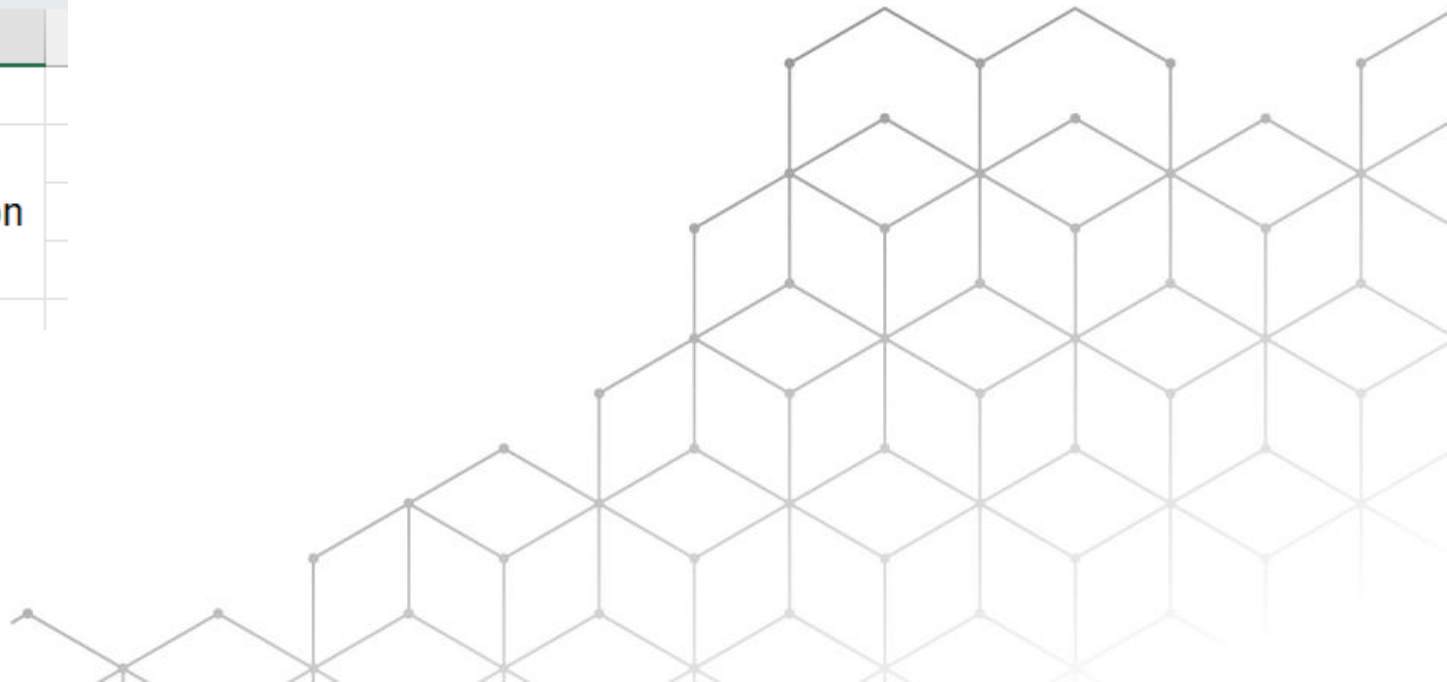
1. **Tabular Format** (rows and columns)
2. **Consistent Schema** (each column has a type)
3. **Easy to Query** (filter, sort, and analyze)



Why it is Semi-structured

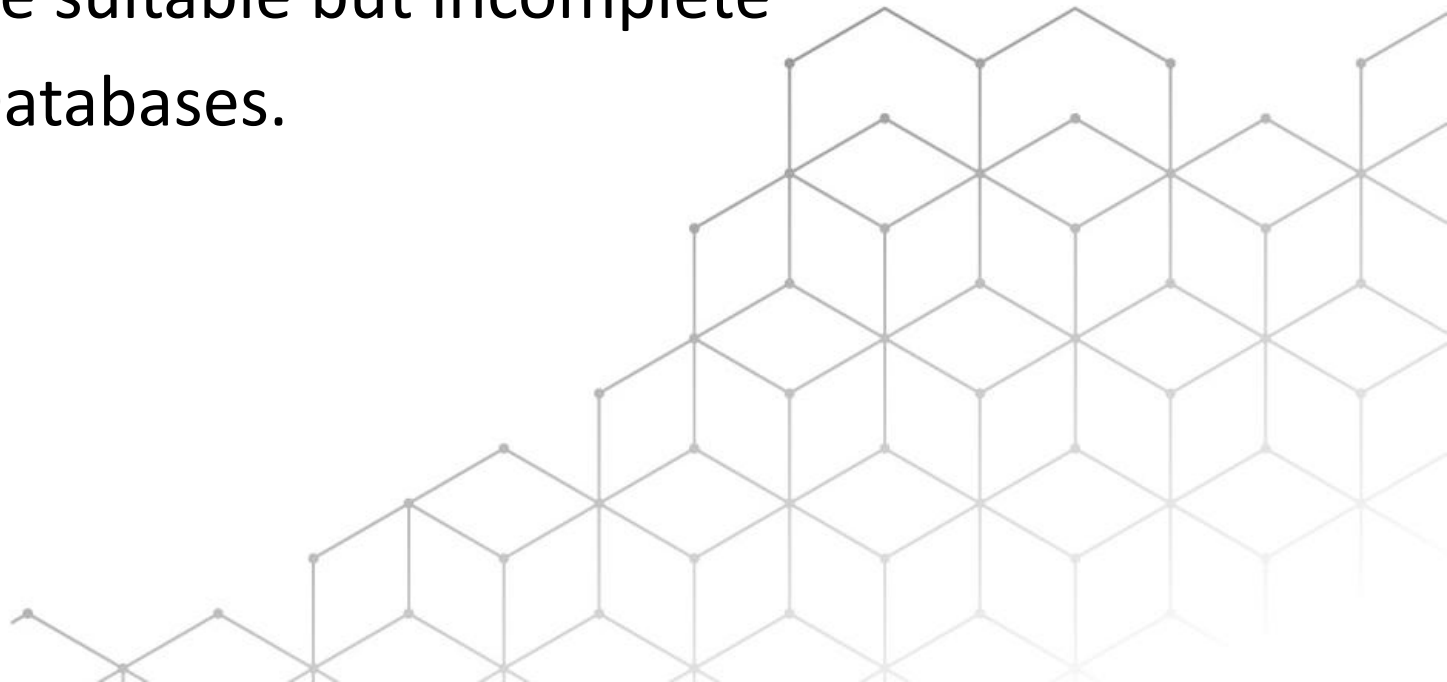
Content without structure – If the spreadsheet contains **merged cells**, **embedded images**, **mixed type columns**, or **free-text paragraphs**, it can be considered semi-structured.

	A	B
1	Data	Other Data
2	Maria	Long paragraph with lots of information
3	26	
4	23.4	
5		



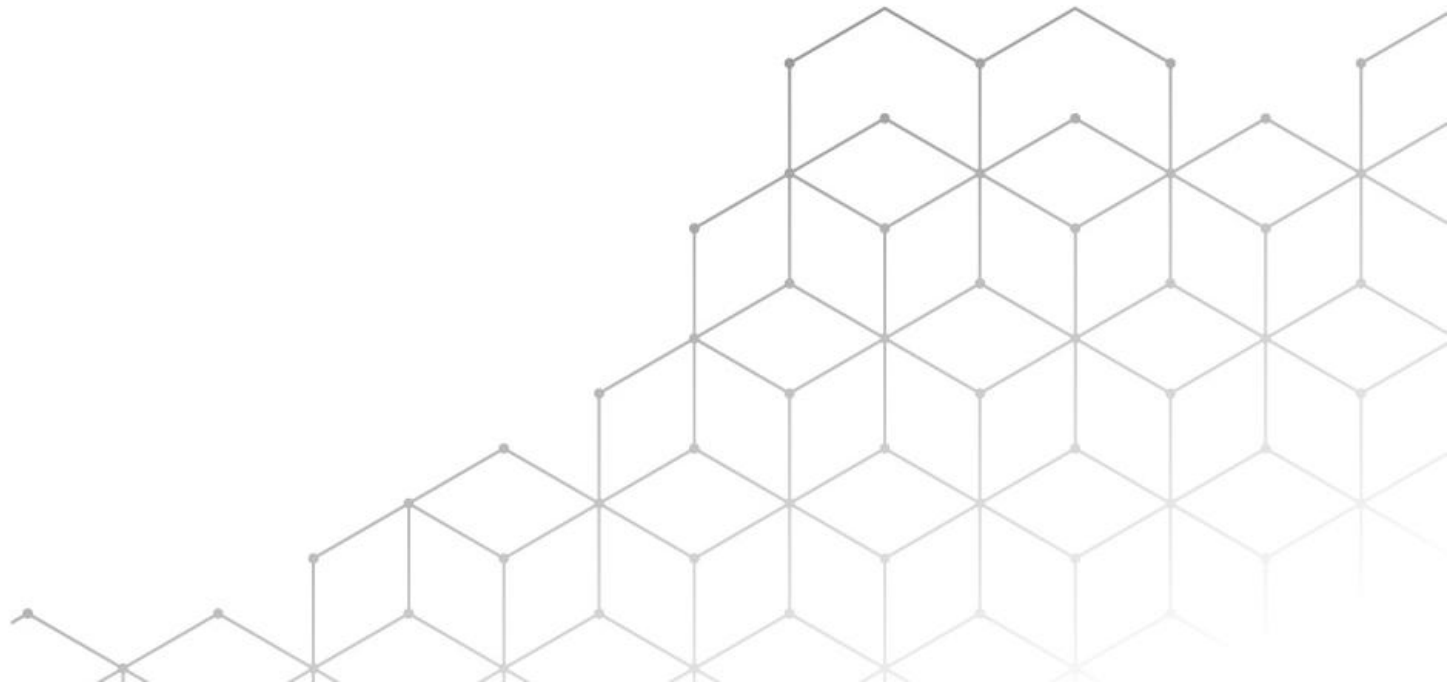
Semi-structured Data (Properties I)

- Partially structured
- Records can have some consistent and defined fields
- However, each field is not guaranteed to be present in all records
- Tabular representation can be suitable but incomplete
- Does not fit into Relational Databases.



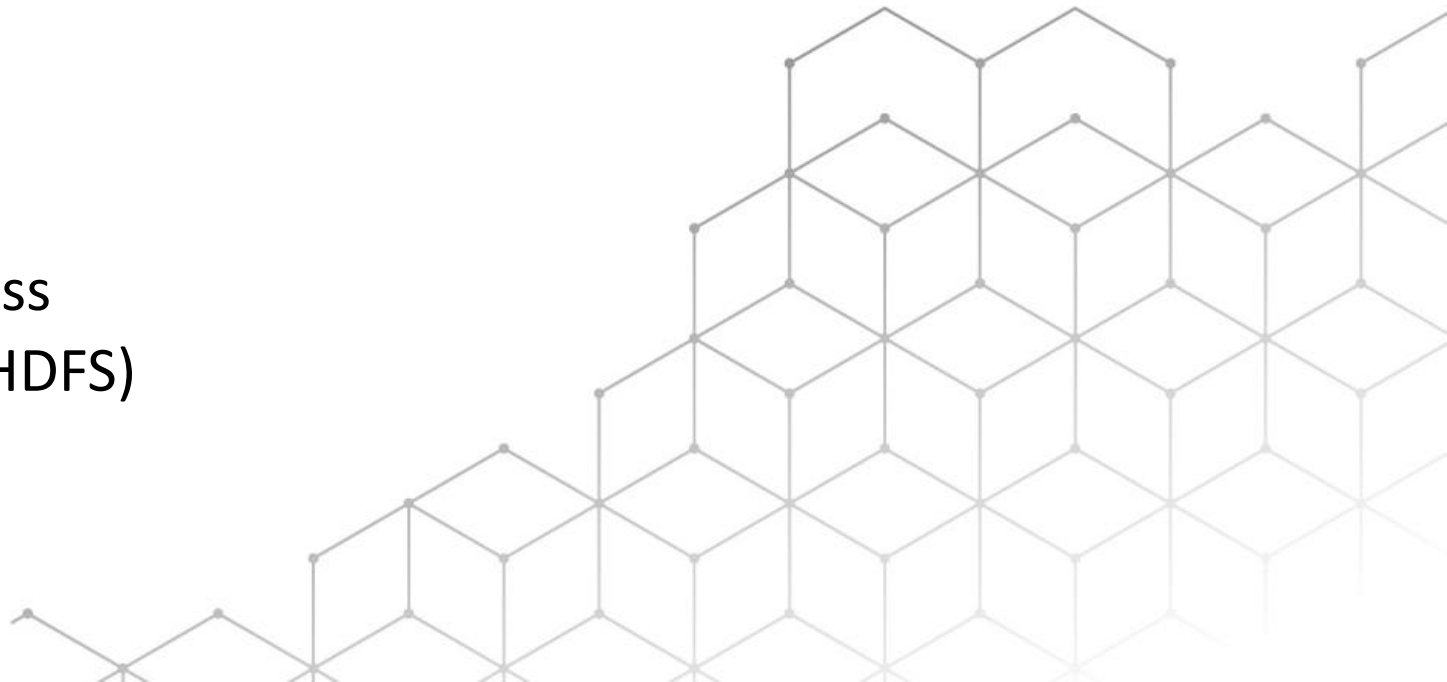
Semi-structured Data (Properties II)

- It can be information sparse
- Can be algorithmically challenging to filter and search
- Quality highly dependent on the source system
- Examples: XML, JSON



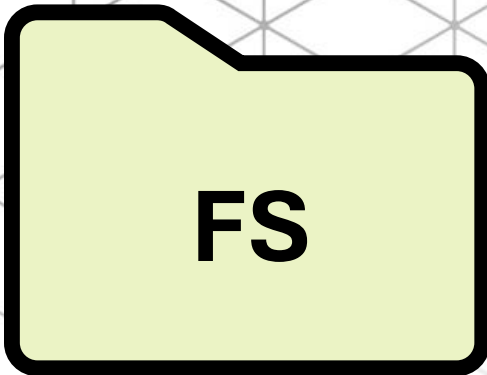
Agenda for Today...

- Types of Data
- Distributed File System (DFS)
 - File System Definition
 - Key Functions of File System
 - Standards
 - Distributed File System (DFS)
- How DFS Works?
- Operations on DFS
- Fault Tolerance by Replication
- Concurrency and Multi-User Access
- Hadoop Distributed File System (HDFS)
- Databricks File System (DBFS)



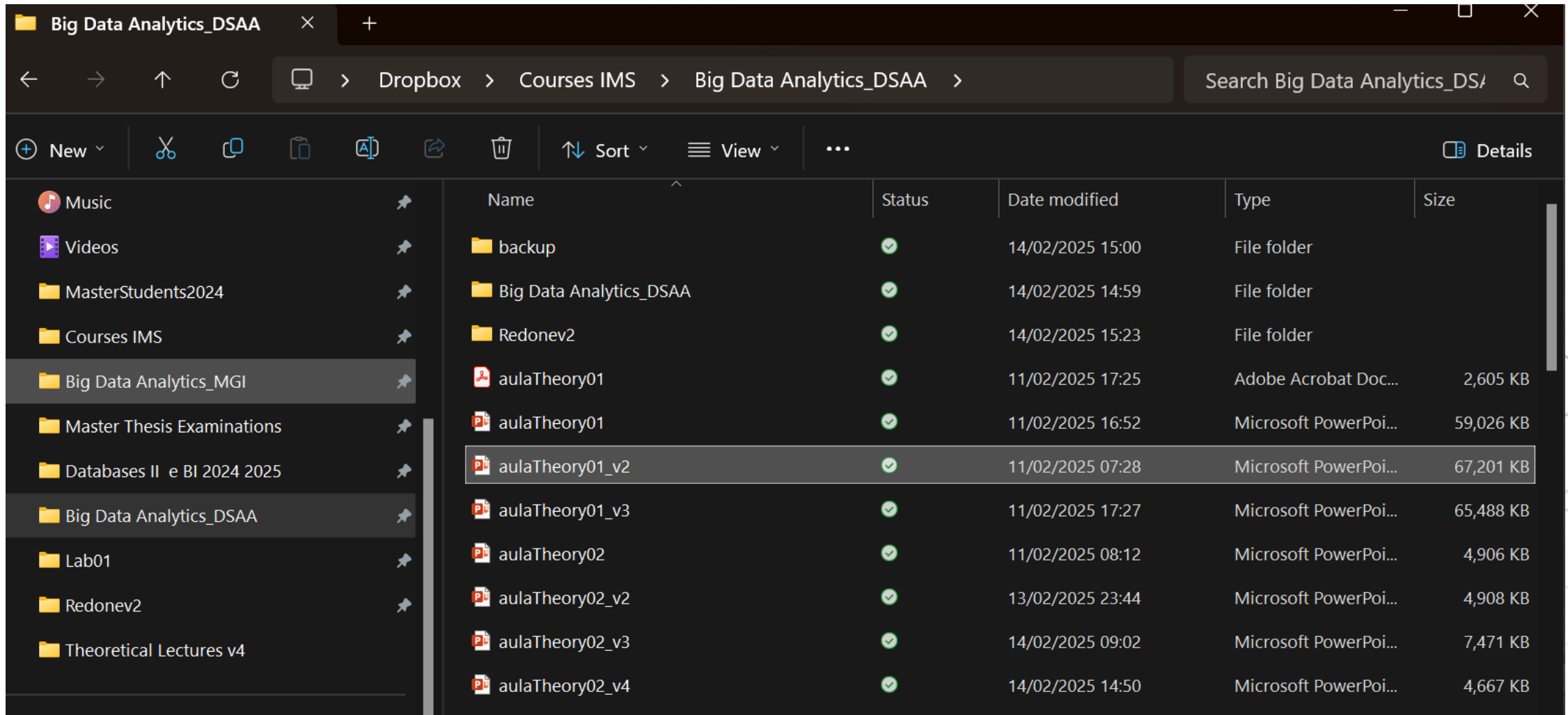
File System

- A **file system** is a **method and structure used by an operating system (OS) to manage, store, organize, and retrieve data on storage devices** such as hard drives, SSDs, and external storage.
- It provides a way to create, read, write, modify, and delete files, ensuring efficient data access and management.



FS

Example of File System



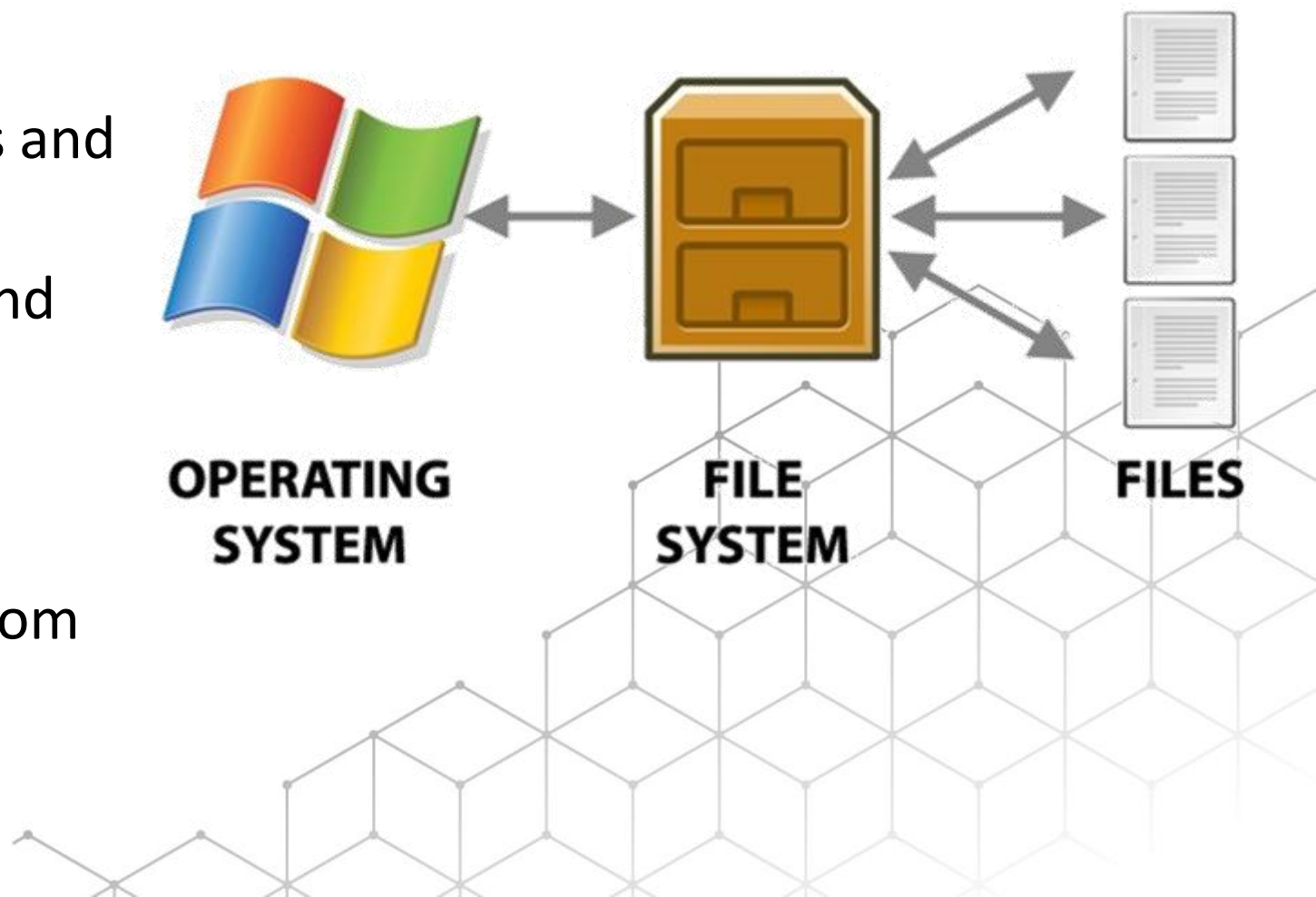
The screenshot shows a Windows File Explorer window with the following details:

- Address Bar:** Dropbox > Courses IMS > Big Data Analytics_DSAA
- Search Bar:** Search Big Data Analytics_DSAA
- Left Sidebar:** A list of folders including Music, Videos, MasterStudents2024, Courses IMS, Big Data Analytics_MGI, Master Thesis Examinations, Databases II e BI 2024 2025, Big Data Analytics_DSAA (selected), Lab01, Redonev2, and Theoretical Lectures v4.
- Main Pane:** A table listing files and folders with columns: Name, Status, Date modified, Type, and Size.

Name	Status	Date modified	Type	Size
backup	✓	14/02/2025 15:00	File folder	
Big Data Analytics_DSAA	✓	14/02/2025 14:59	File folder	
Redonev2	✓	14/02/2025 15:23	File folder	
aulaTheory01	✓	11/02/2025 17:25	Adobe Acrobat Doc...	2,605 KB
aulaTheory01	✓	11/02/2025 16:52	Microsoft PowerPoi...	59,026 KB
aulaTheory01_v2	✓	11/02/2025 07:28	Microsoft PowerPoi...	67,201 KB
aulaTheory01_v3	✓	11/02/2025 17:27	Microsoft PowerPoi...	65,488 KB
aulaTheory02	✓	11/02/2025 08:12	Microsoft PowerPoi...	4,906 KB
aulaTheory02_v2	✓	13/02/2025 23:44	Microsoft PowerPoi...	4,908 KB
aulaTheory02_v3	✓	14/02/2025 09:02	Microsoft PowerPoi...	7,471 KB
aulaTheory02_v4	✓	14/02/2025 14:50	Microsoft PowerPoi...	4,667 KB

Key Functions of File System

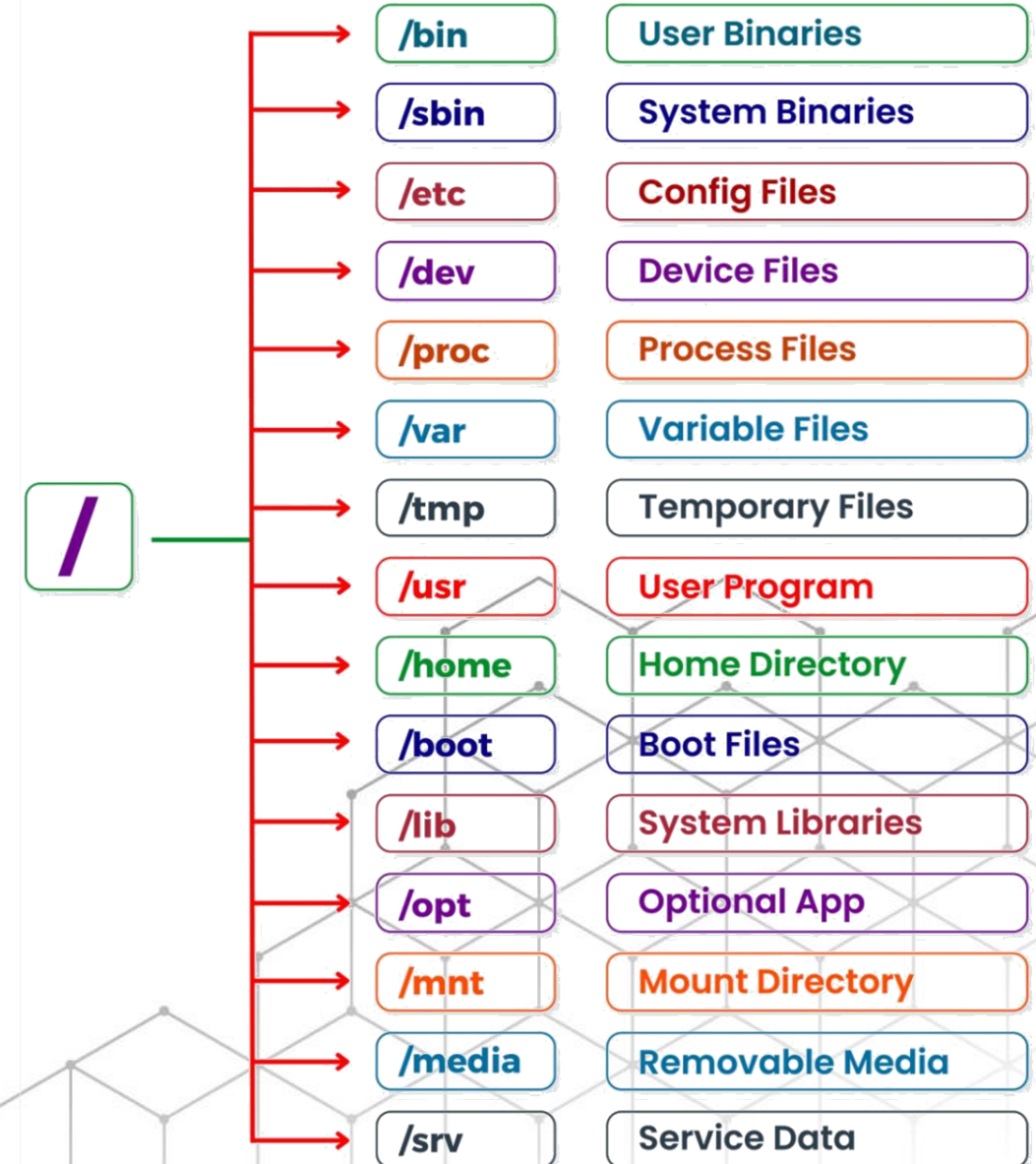
- File Organization – Arranges data in files and directories in a hierarchy.
- Storage Management – Allocates and tracks space usage.
- Access – Manages permissions and security for different users.
- Data Retrieval – Enables search, reading, and modification.
- Fault Tolerance – Recover data from failures and prevent corruption.



Standards

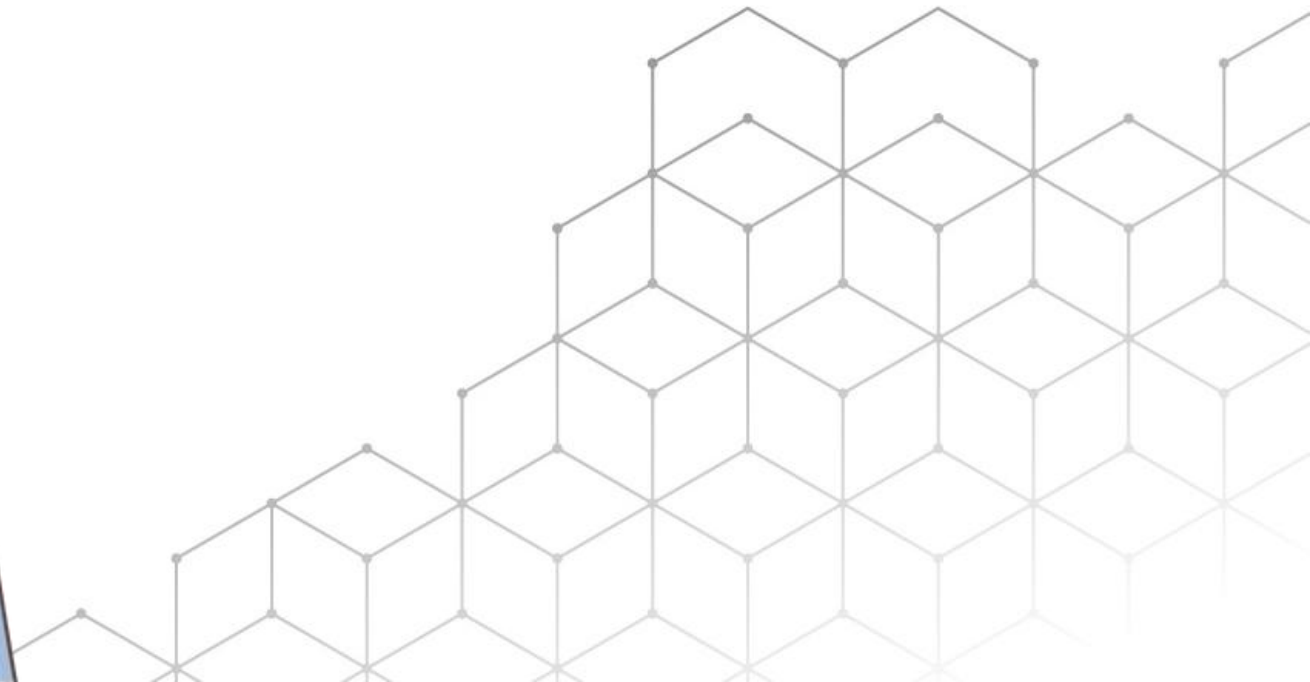
- There are standards such as **Filesystem Hierarchy Standard (FHS)** is a reference describing the conventions used for the layout of Unix-like systems.

LINUX FILE SYSTEM



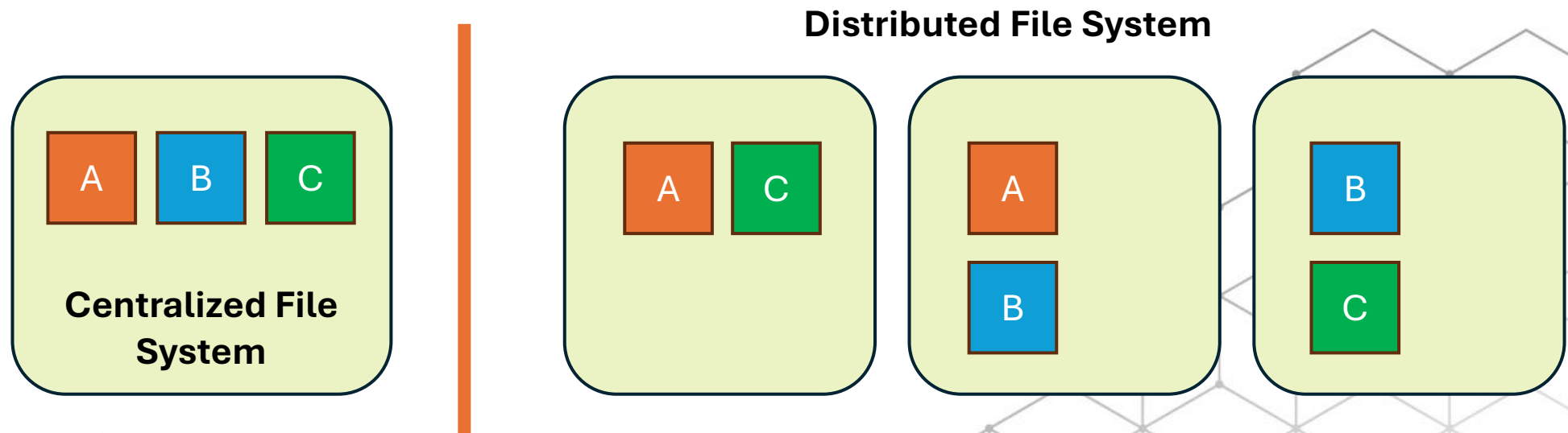
Distributed Database

- A distributed file system is a FS that is spread (“distributed”) across multiple machines/computers
- For an end-user, interacting with a distributed file system should be indistinguishable from a non-distributed one



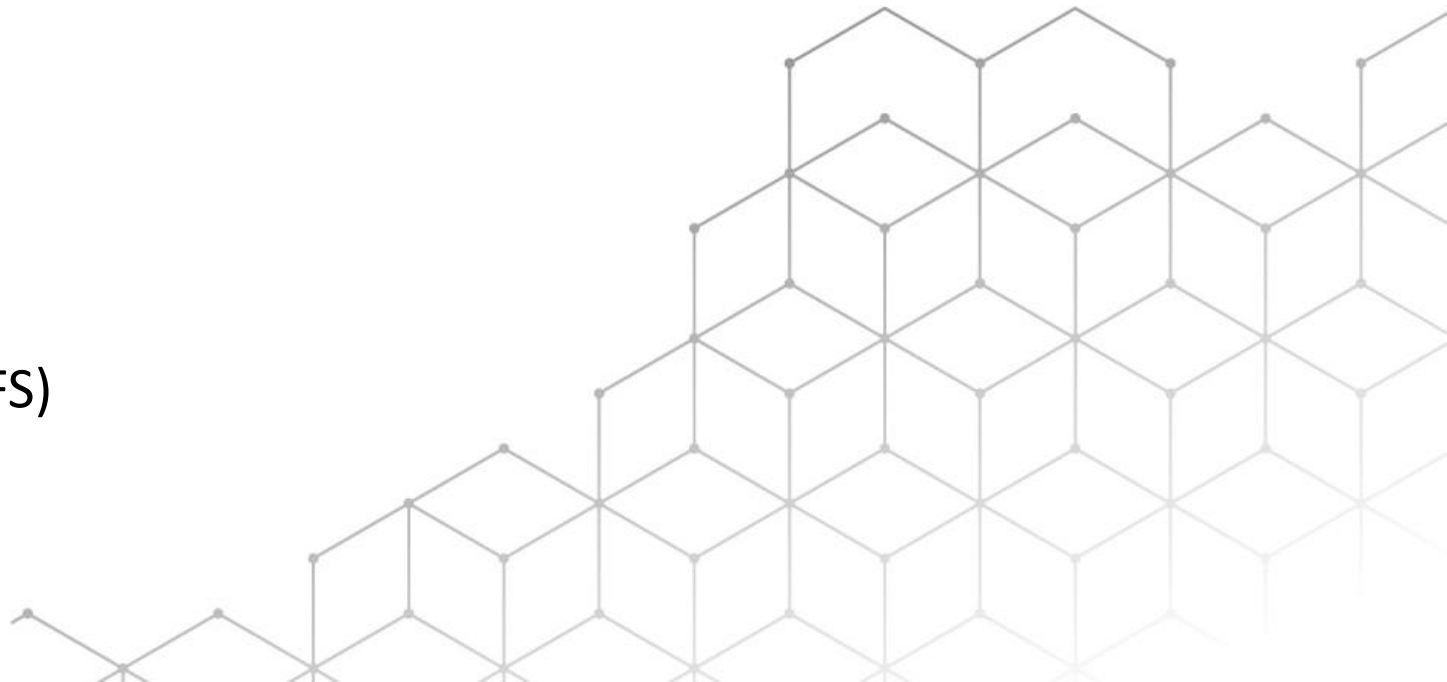
Distributed File System

- A Distributed File System (DFS) is a file system that allows files to be stored across multiple nodes while appearing as a single, unified file system to users and applications.



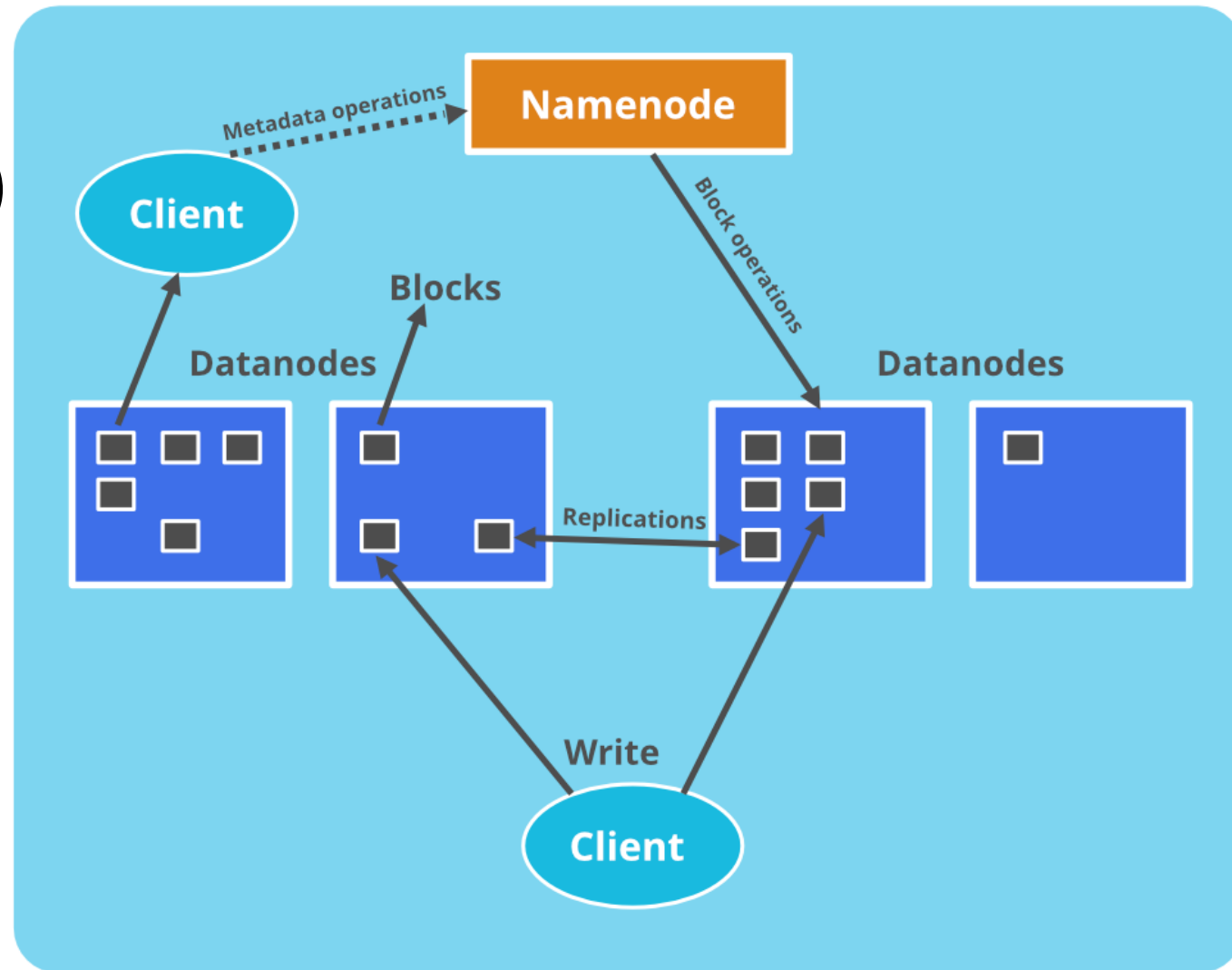
Agenda for Today...

- Types of Data
- Distributed File System (DFS)
- How DFS Works?
 - Data Nodes
 - Metadata Server
 - Clients
 - Replication on DFS
 - Operations on DFS
- Operations on DFS
- Fault Tolerance by Replication
- Concurrency and Multi-User Access
- Hadoop Distributed File System (HDFS)
- Databricks File System (DBFS)



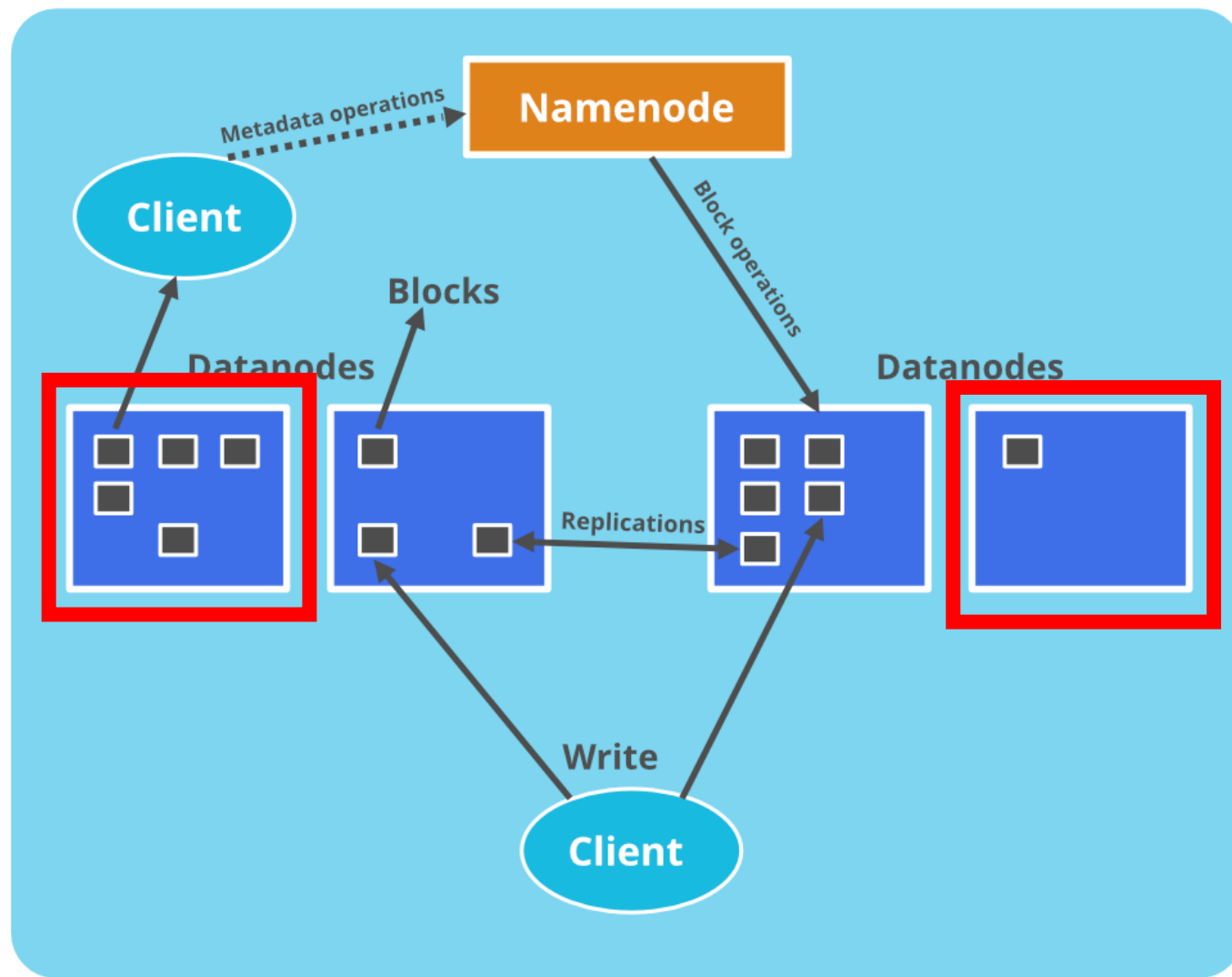
How does DFS Works?

- Data (Storage) Nodes
- Metadata Server (Namenode)
- Clients
- Replication Mechanism



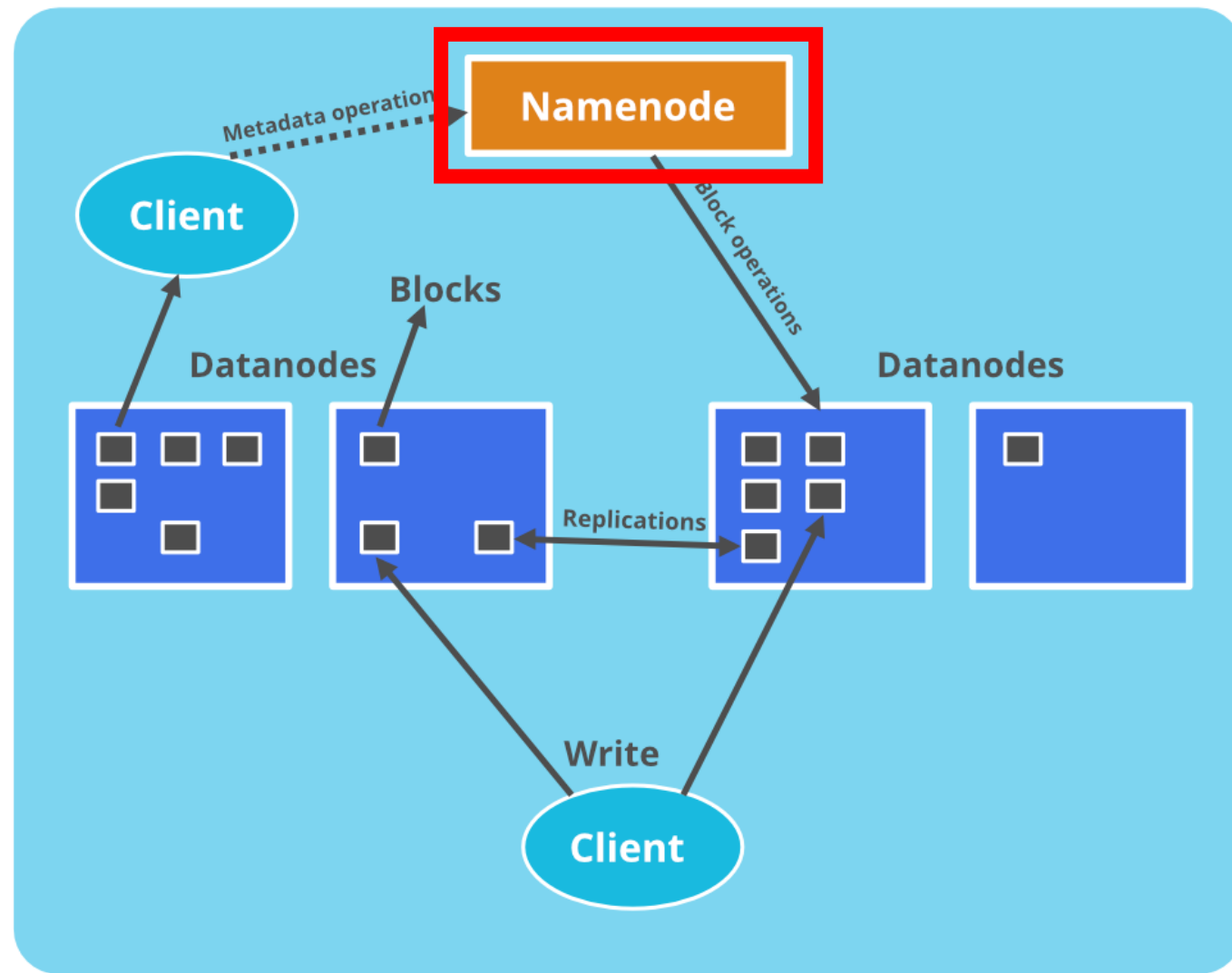
Storage Node

- Servers or machines where data is physically stored. Each node typically contains multiple file blocks.



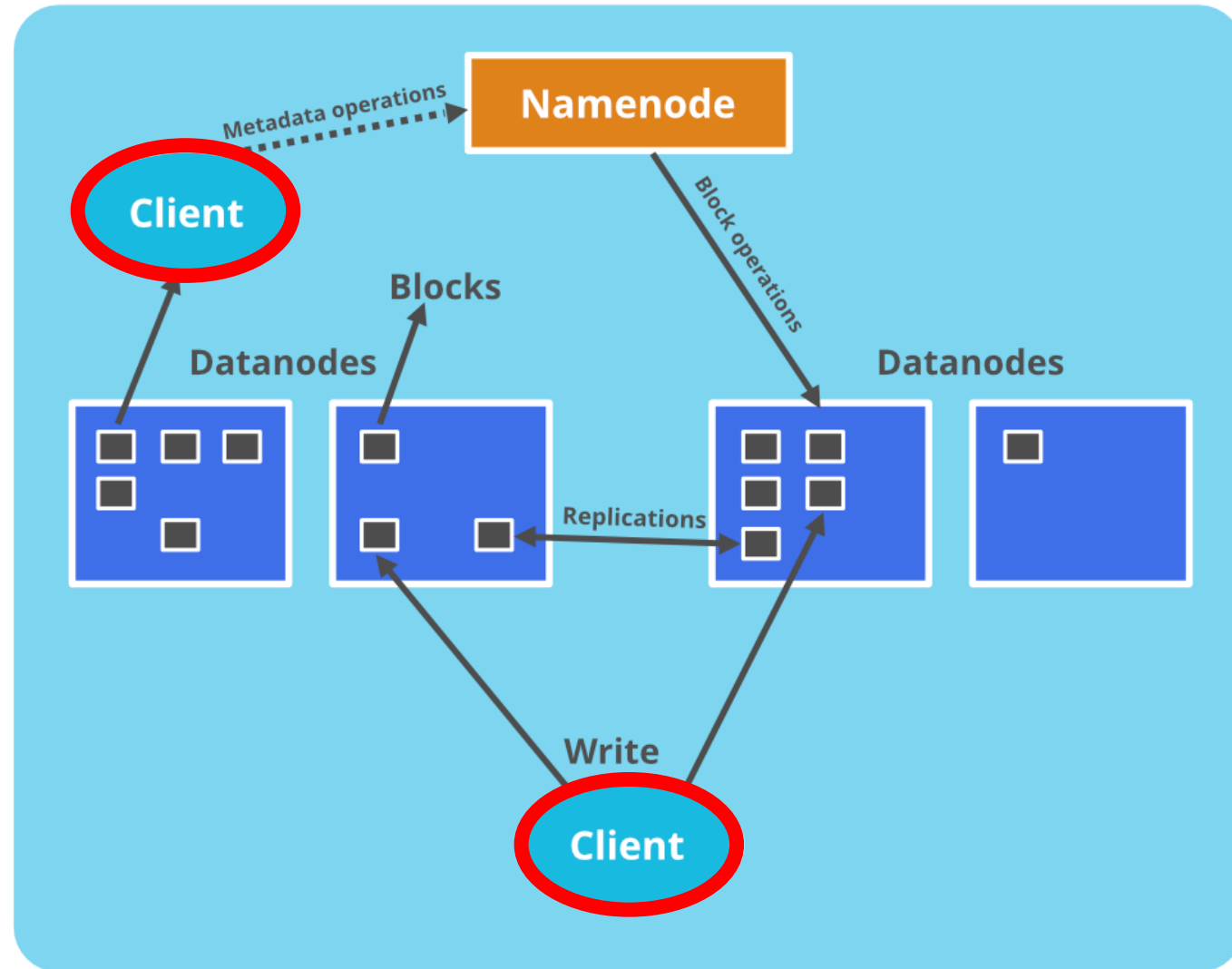
Metadata Server

- Manages file locations, metadata, and access control but does not store actual file data.



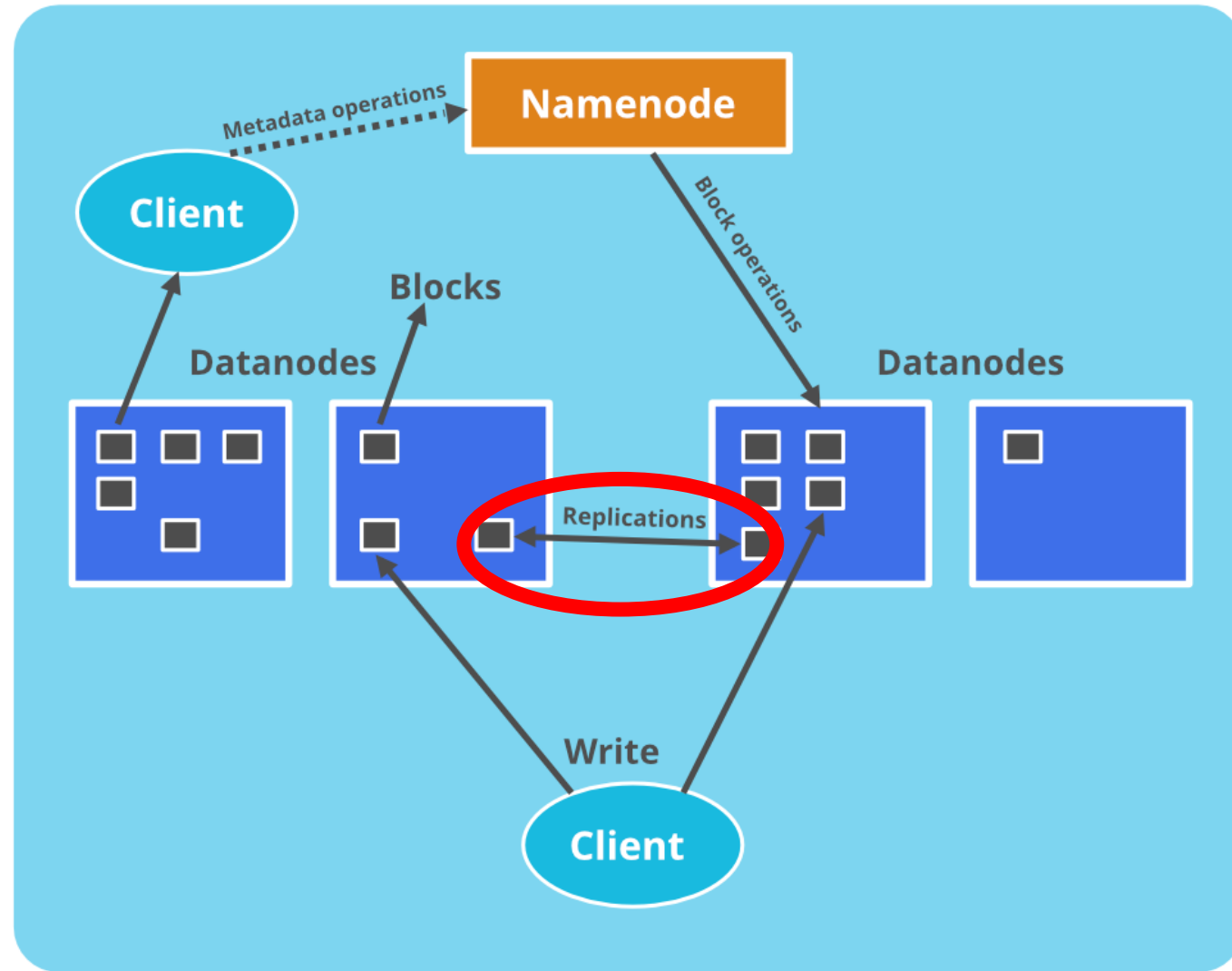
Clients

- Users or applications that request, read, or write files. The client interacts with the DFS without knowing the actual data location



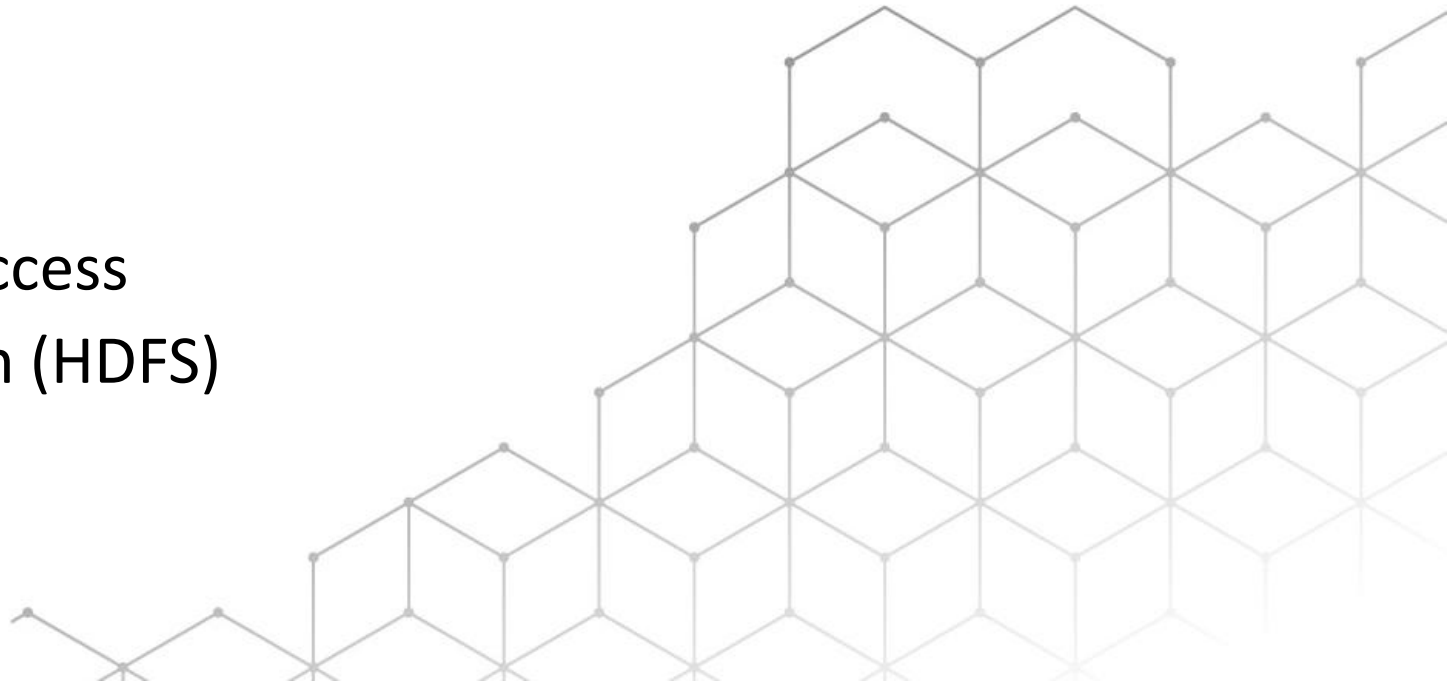
Replication Mechanism

- Ensures data redundancy by storing multiple copies across different nodes to prevent data loss in case of failures.



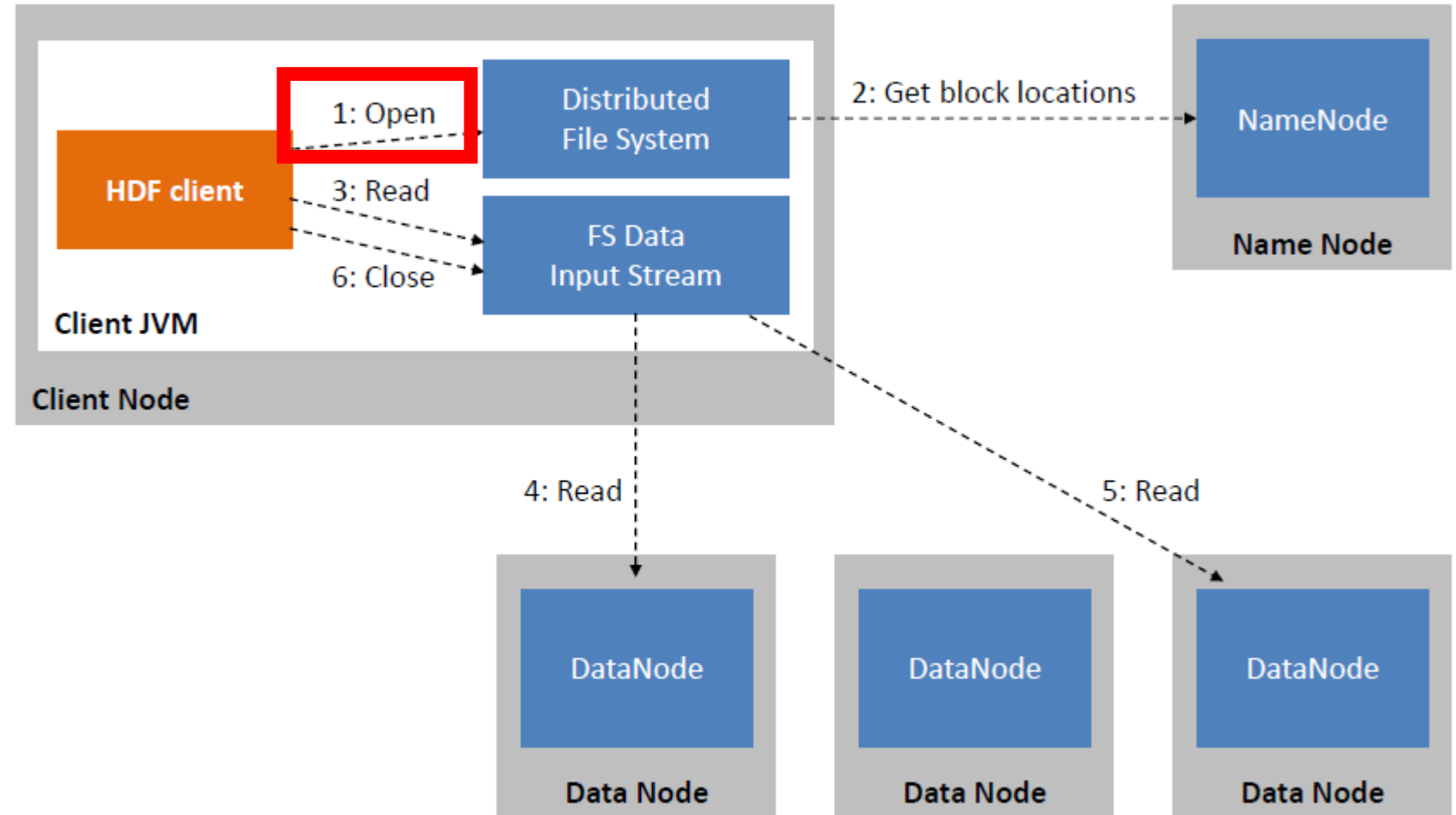
Agenda for Today...

- Types of Data
- Distributed File System (DFS)
- How DFS Works?
- Operations on DFS
 - File Access & Retrieval
 - File Creation & Distribution
- Fault Tolerance by Replication
- Concurrency and Multi-User Access
- Hadoop Distributed File System (HDFS)
- Databricks File System (DBFS)



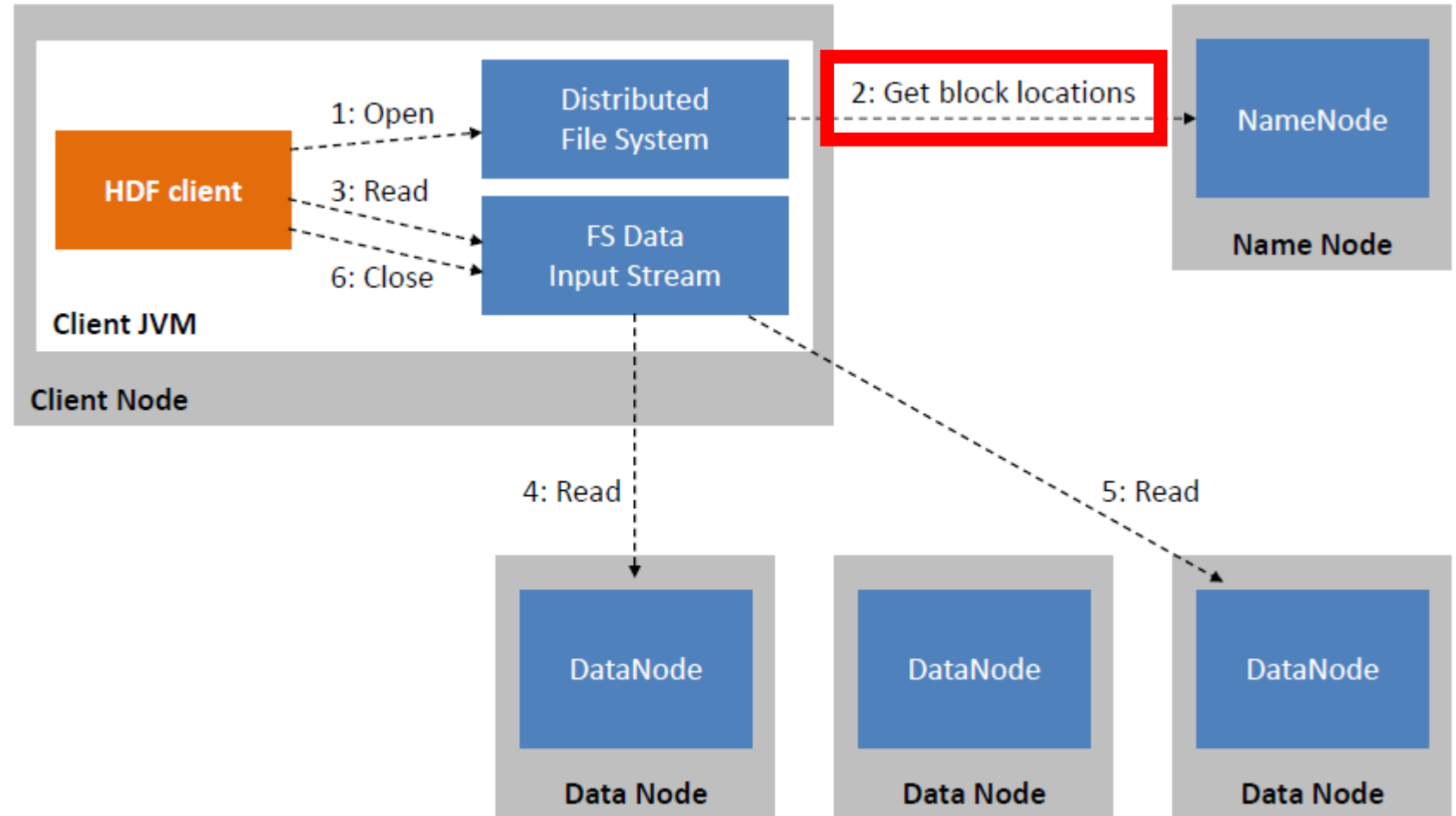
File Access and Retrieval

- 1 – Requests a File



File Access and Retrieval

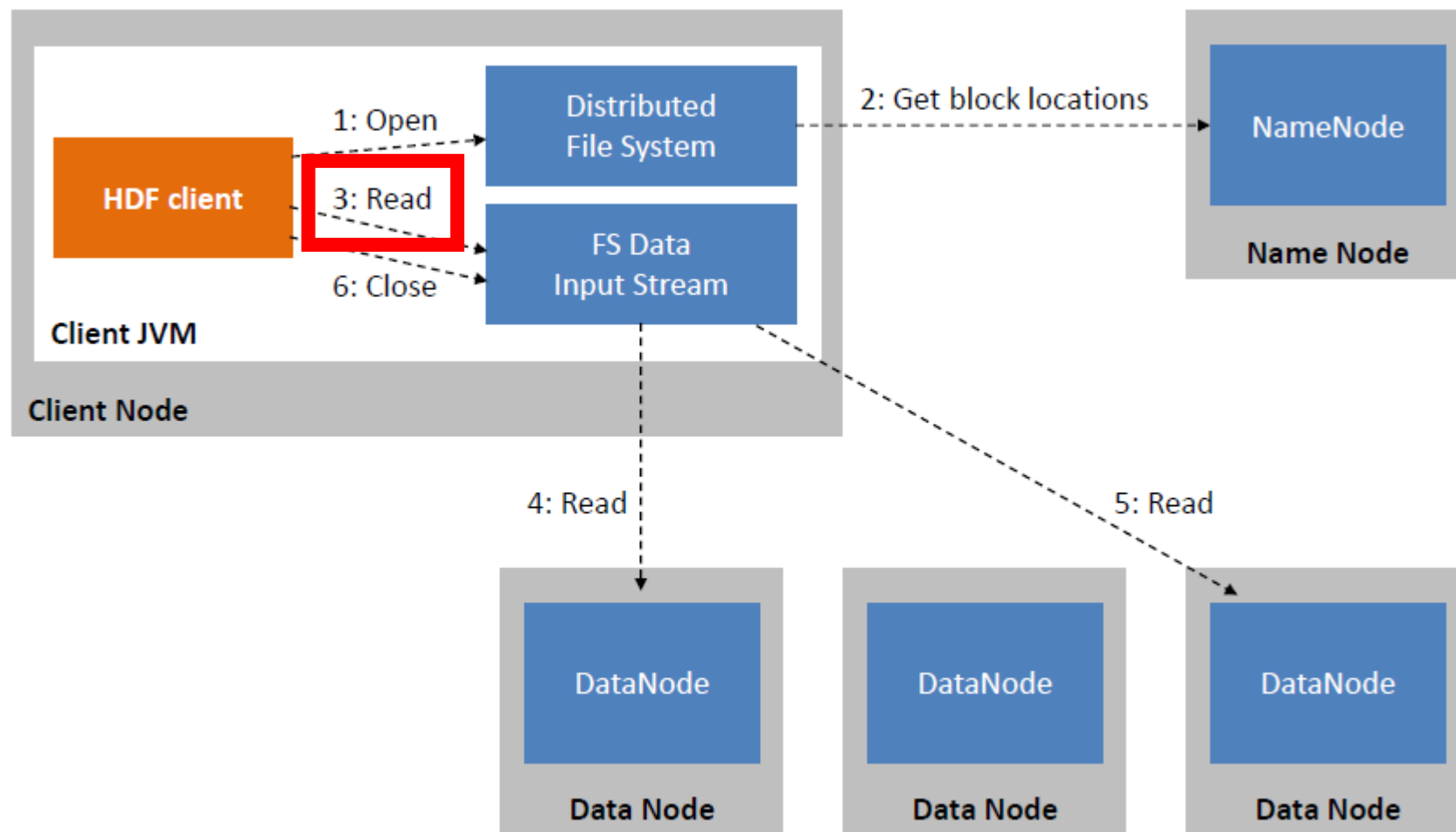
2 – Contact metadata server to map the request to data node locations



File Access and Retrieval

3 - Read

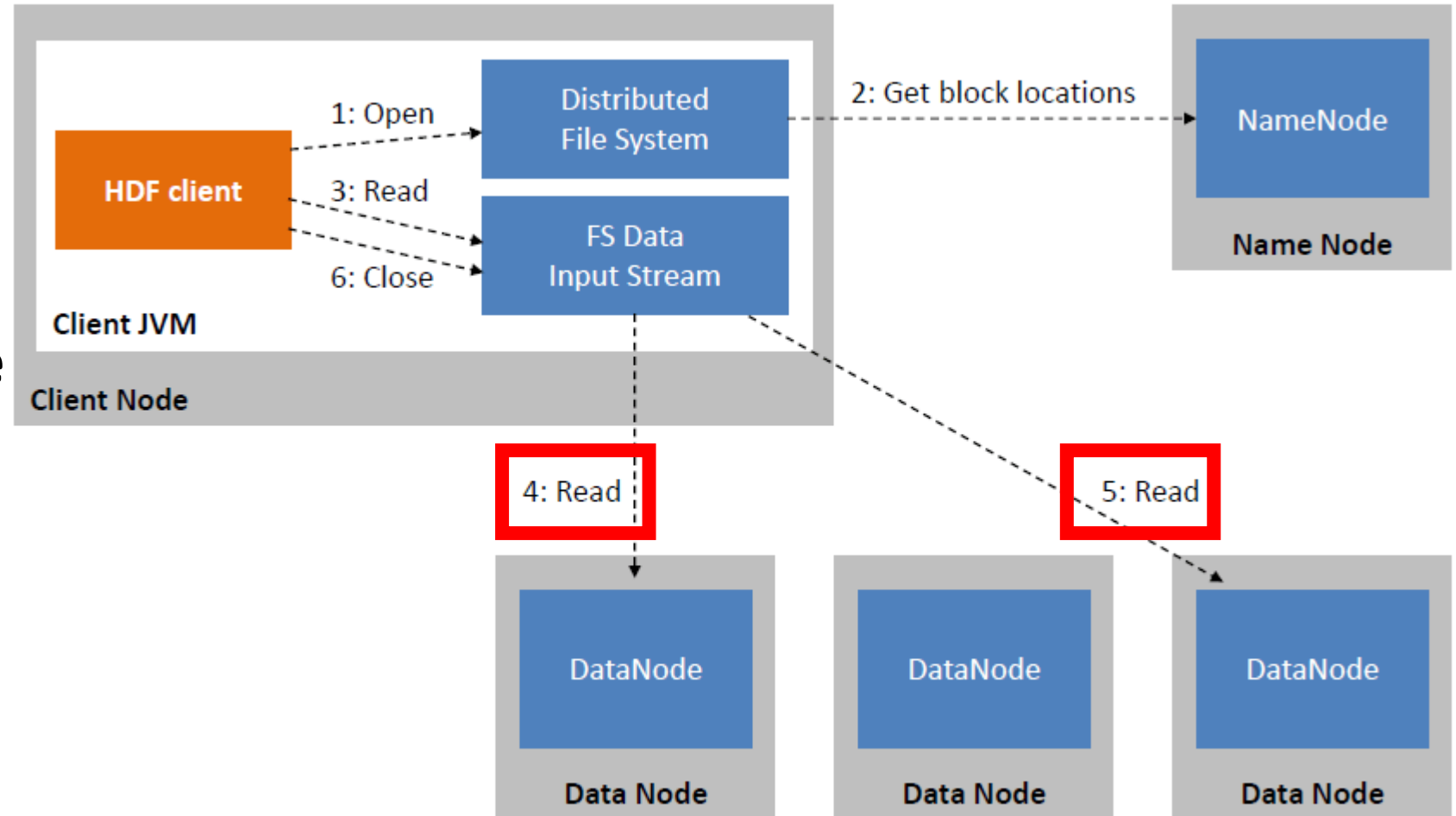
- The client **fetches file blocks directly from the storage nodes**, rather than from a single centralized location.



File Access and Retrieval

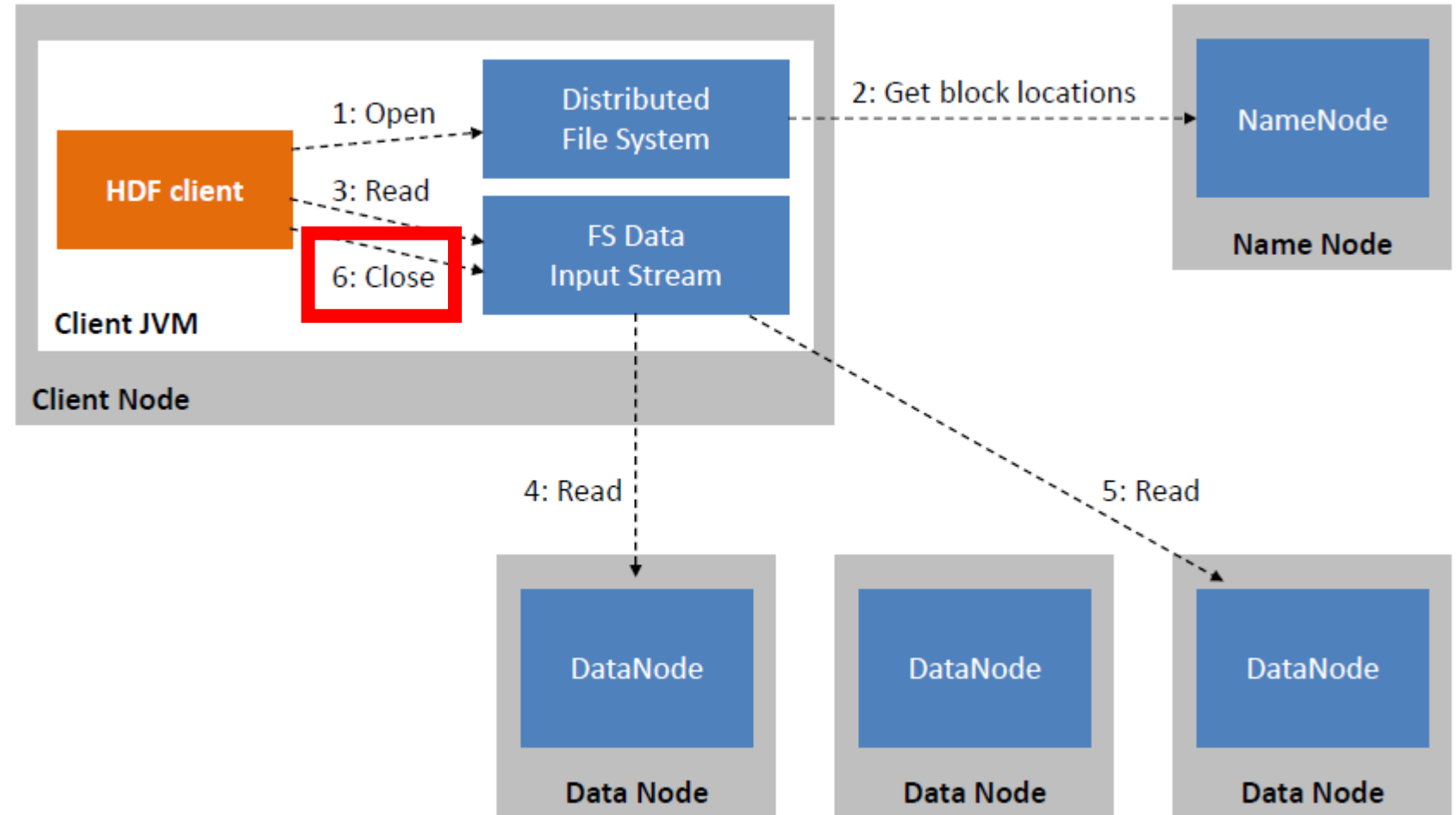
4 & 5 - Read

- The client **fetches file blocks directly from the storage nodes**, rather than from a single centralized location.



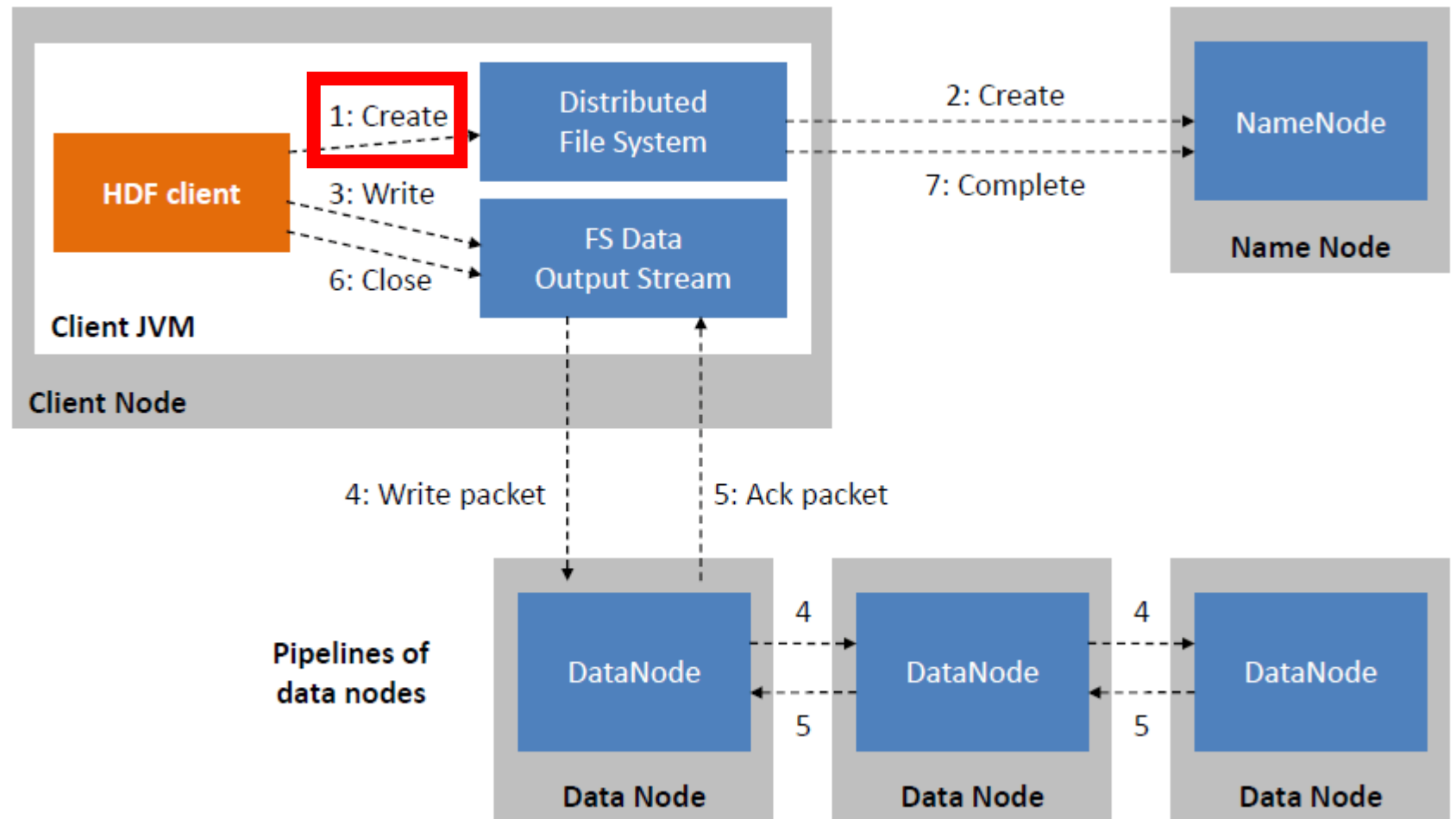
File Access and Retrieval

- 6 - Read
Close the
connection



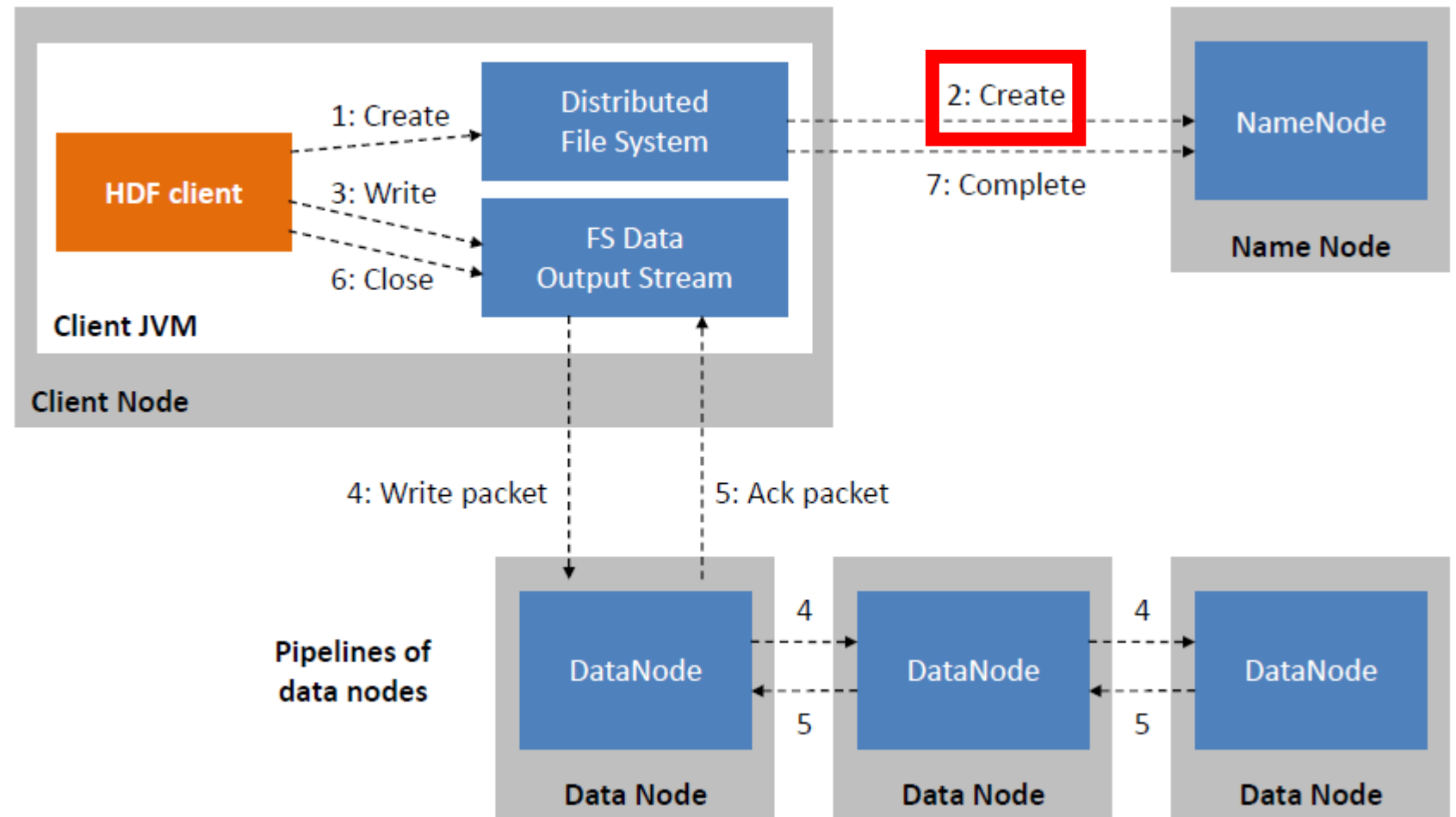
File Storage & Distribution

The client asks the DFS to create a new file



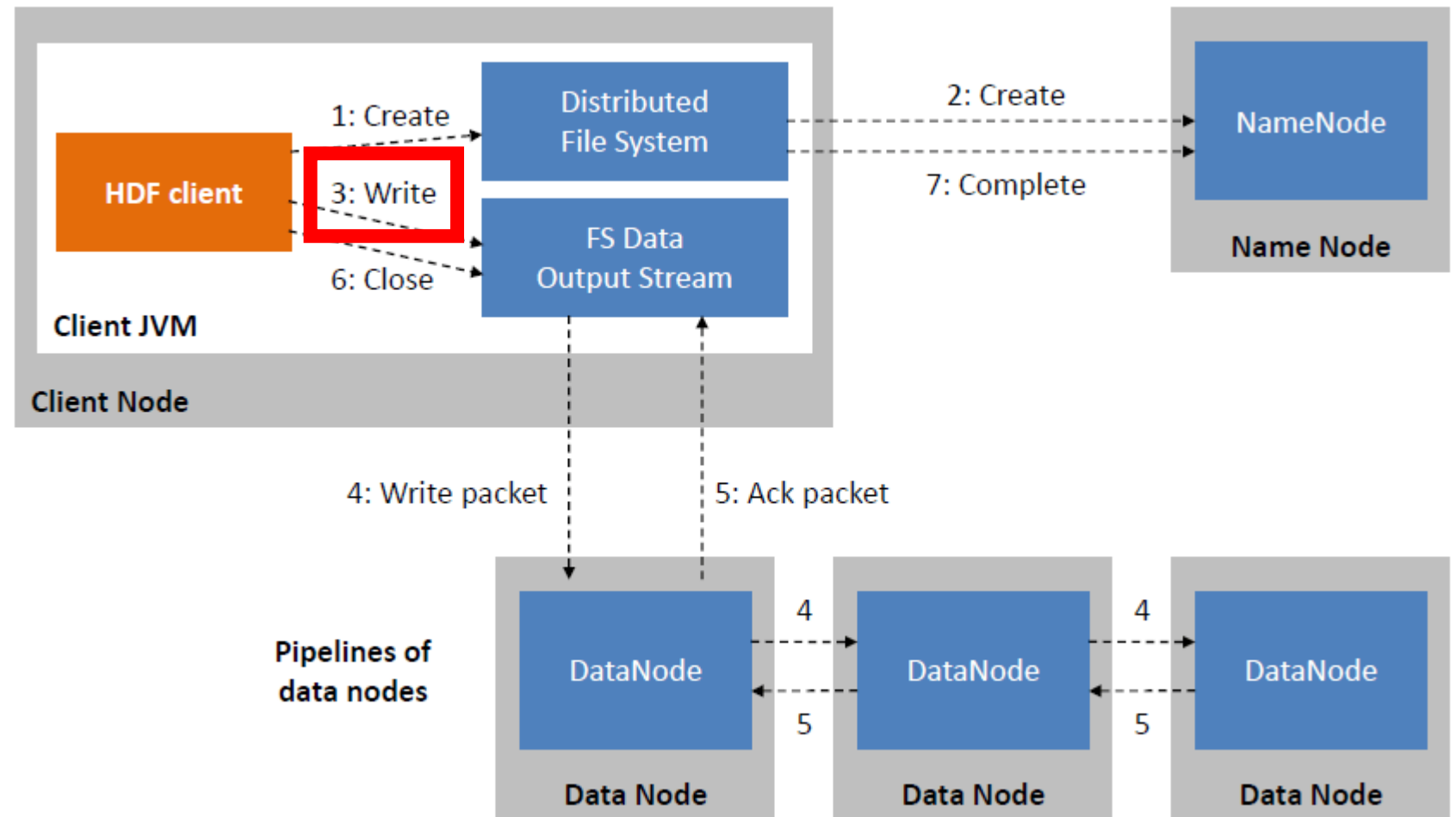
File Storage & Distribution

Create File block
(chunk) location in
Metadata server



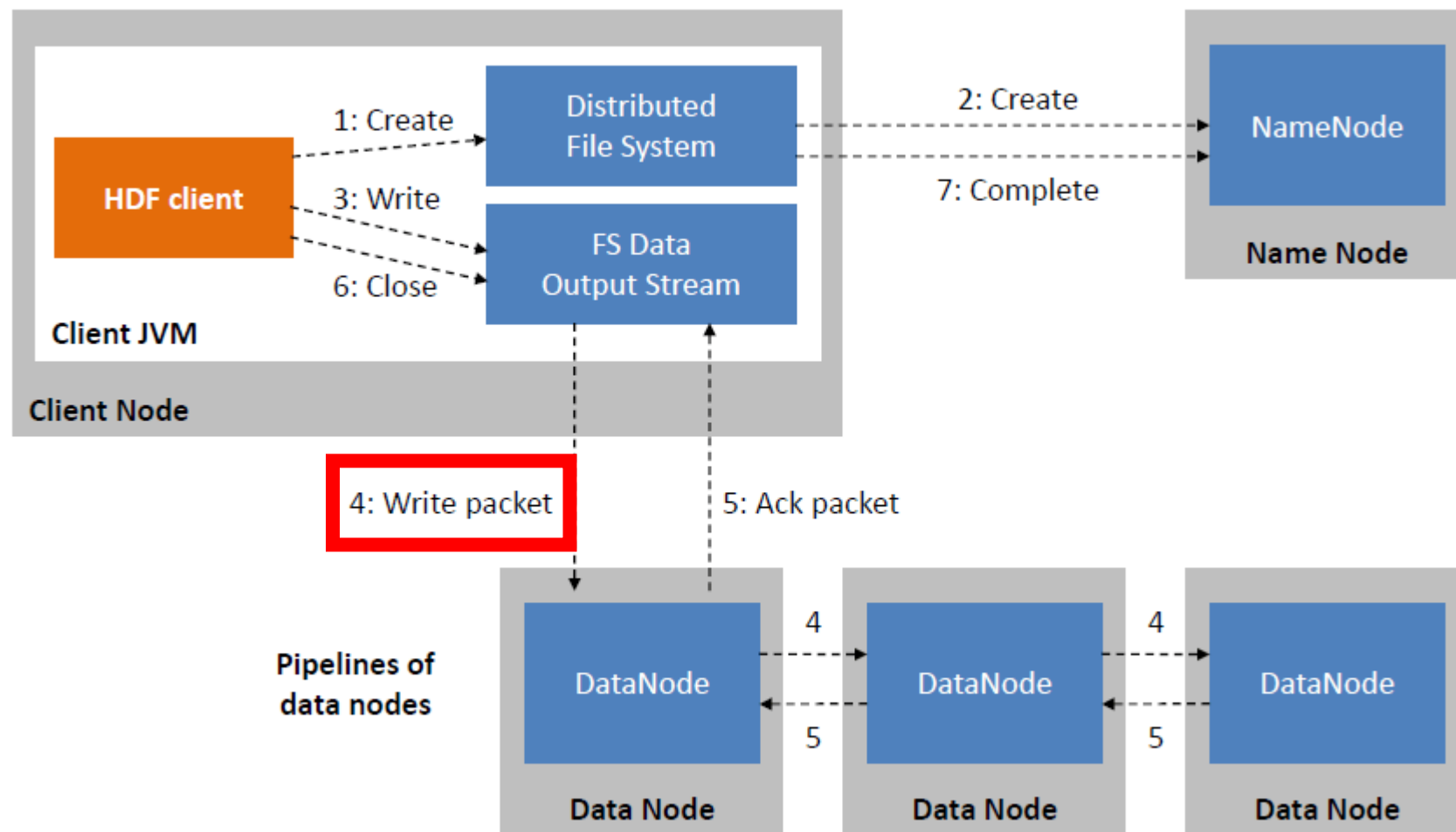
File Storage & Distribution

- The client asks the DFS to perform the write operation



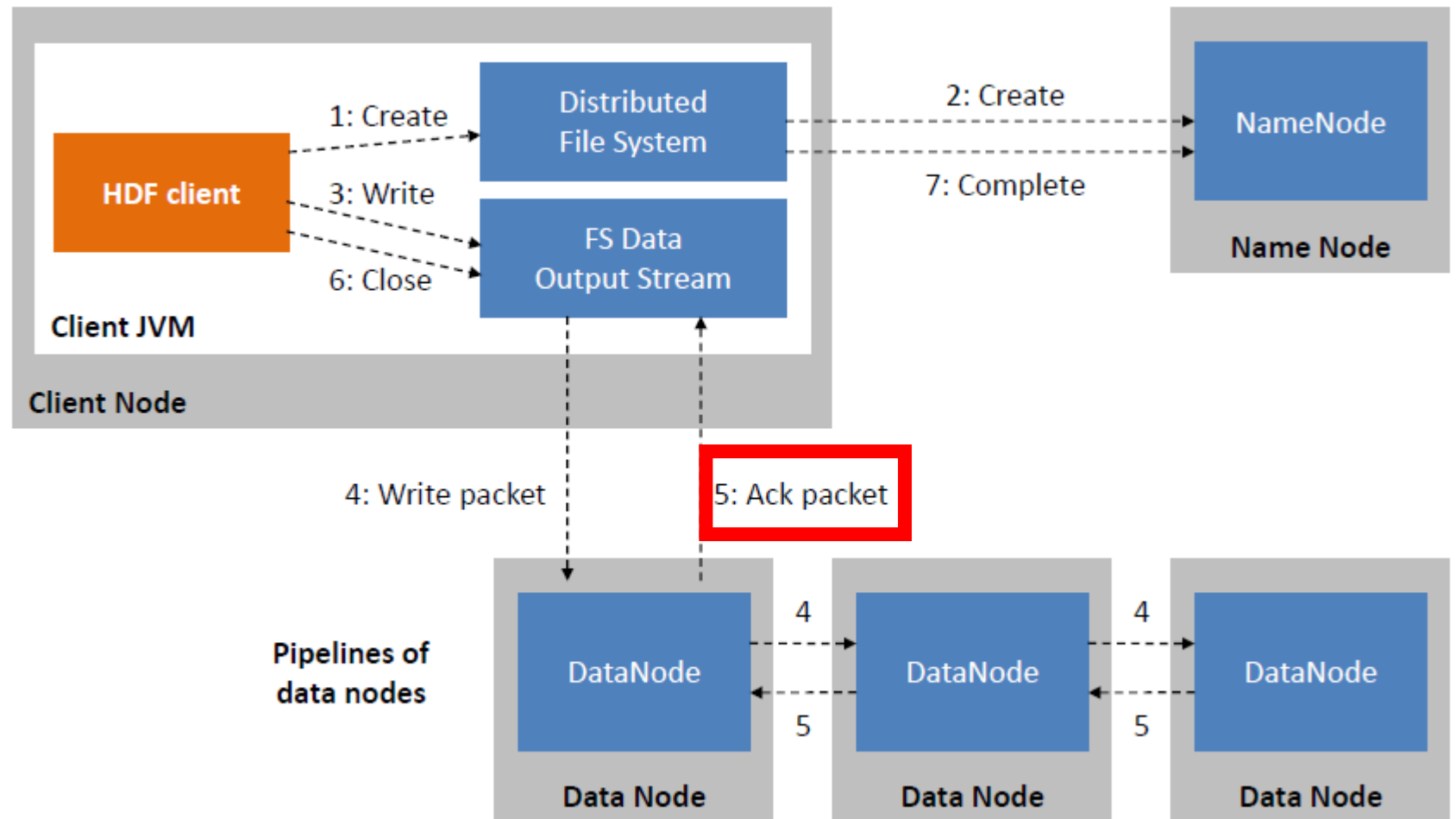
File Storage & Distribution

- Ask one data node to create the block
- Propagate this operation to other nodes
- Replication



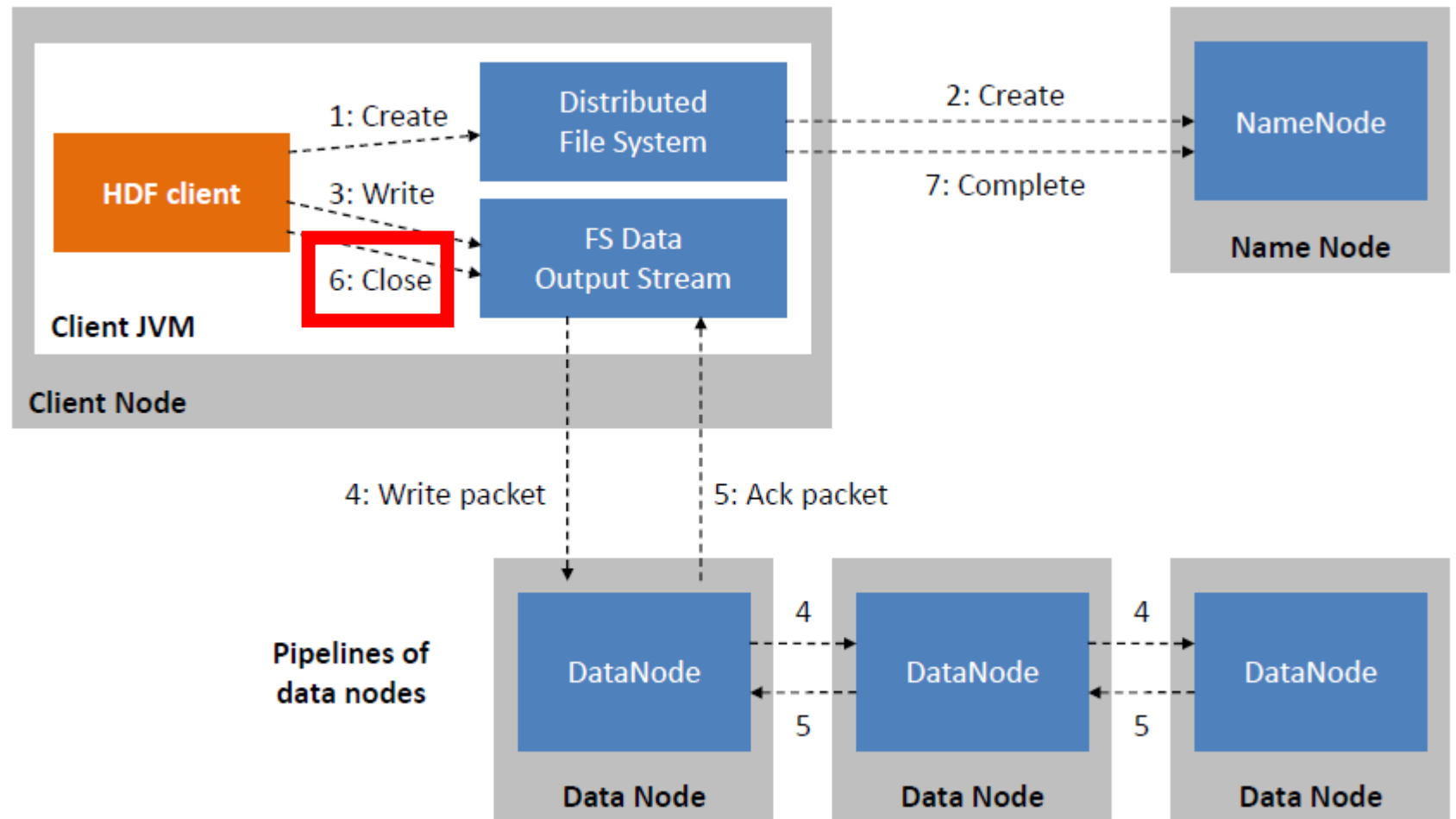
File Storage & Distribution

- Acknowledge the file distribution and block replication



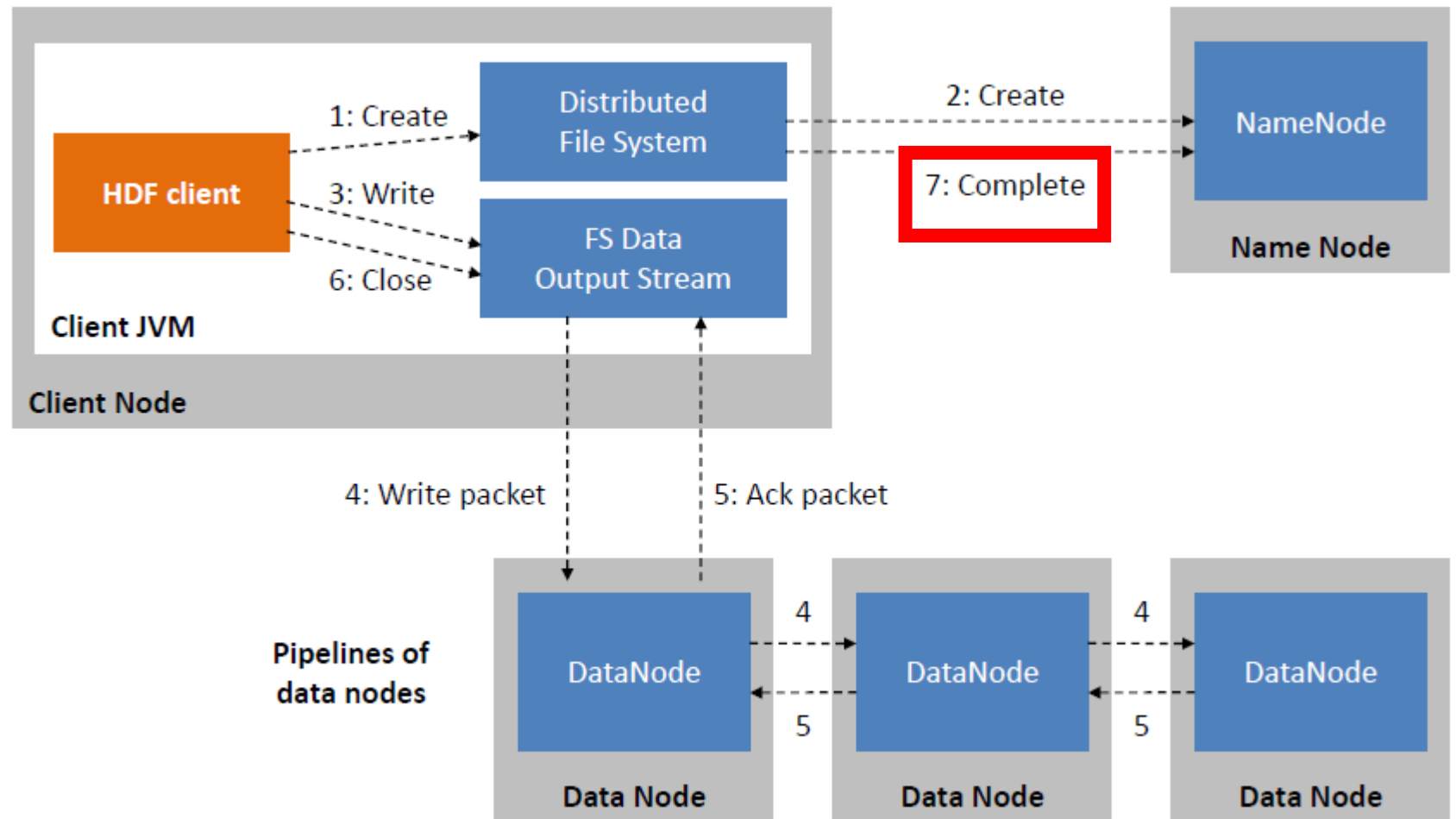
File Storage & Distribution

- Close the output stream



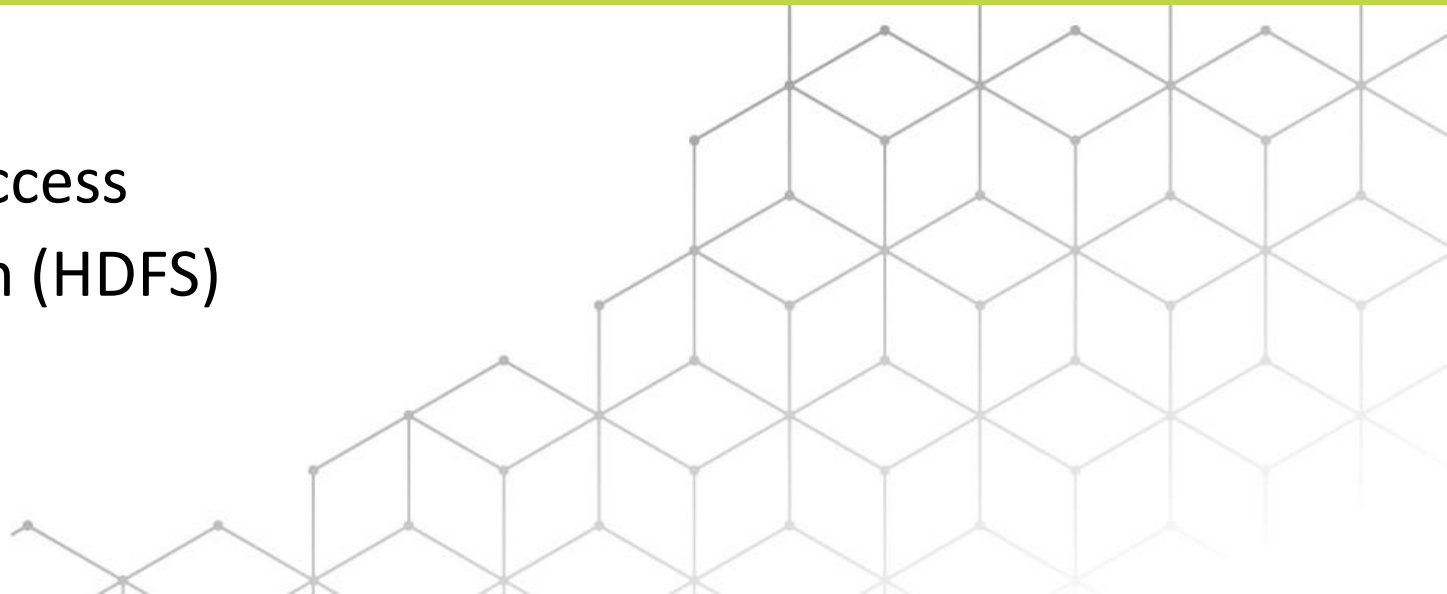
File Storage & Distribution

- Acknowledge the successful operation to namenode



Agenda for Today...

- Types of Data
- Distributed File System (DFS)
- How DFS Works?
- Operations on DFS
- Fault Tolerance by Replication
 - Fault Tolerance
 - Replication
- Concurrency and Multi-User Access
- Hadoop Distributed File System (HDFS)
- Databricks File System (DBFS)



Fault-tolerance

- **Fault tolerance** is related to **availability**, and the two terms are often used interchangeably.
- A system is considered fault tolerant if it continues to work correctly despite the failure of individual components. **Binary outcome**

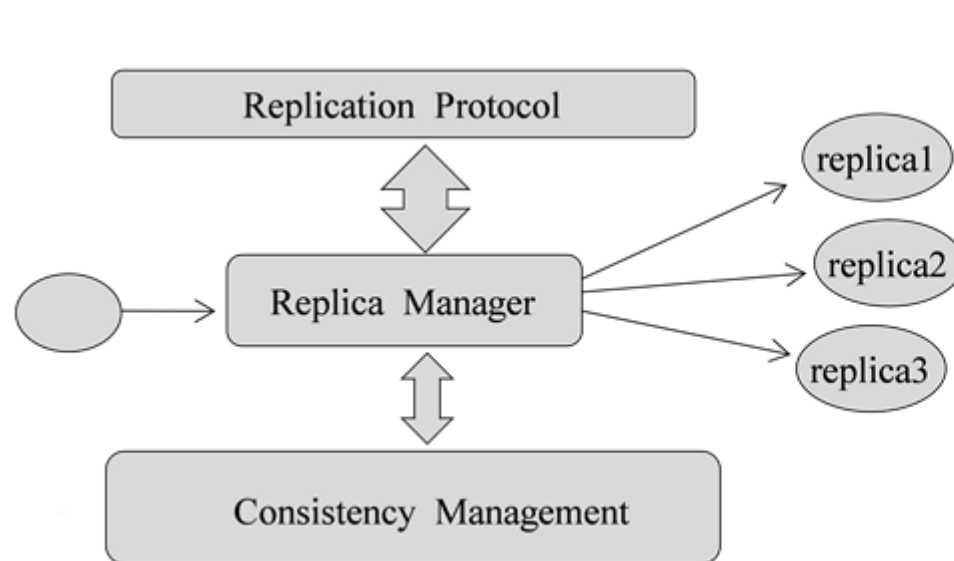
Availability

the percent of time a system spends in a functional state.



Fault Tolerance by Replication

- Replication is the process of **creating and maintaining multiple copies of data across different nodes within a distributed system.**
- This method enhances fault tolerance by ensuring that if one node fails, other nodes retain access to the data, minimizing downtime.

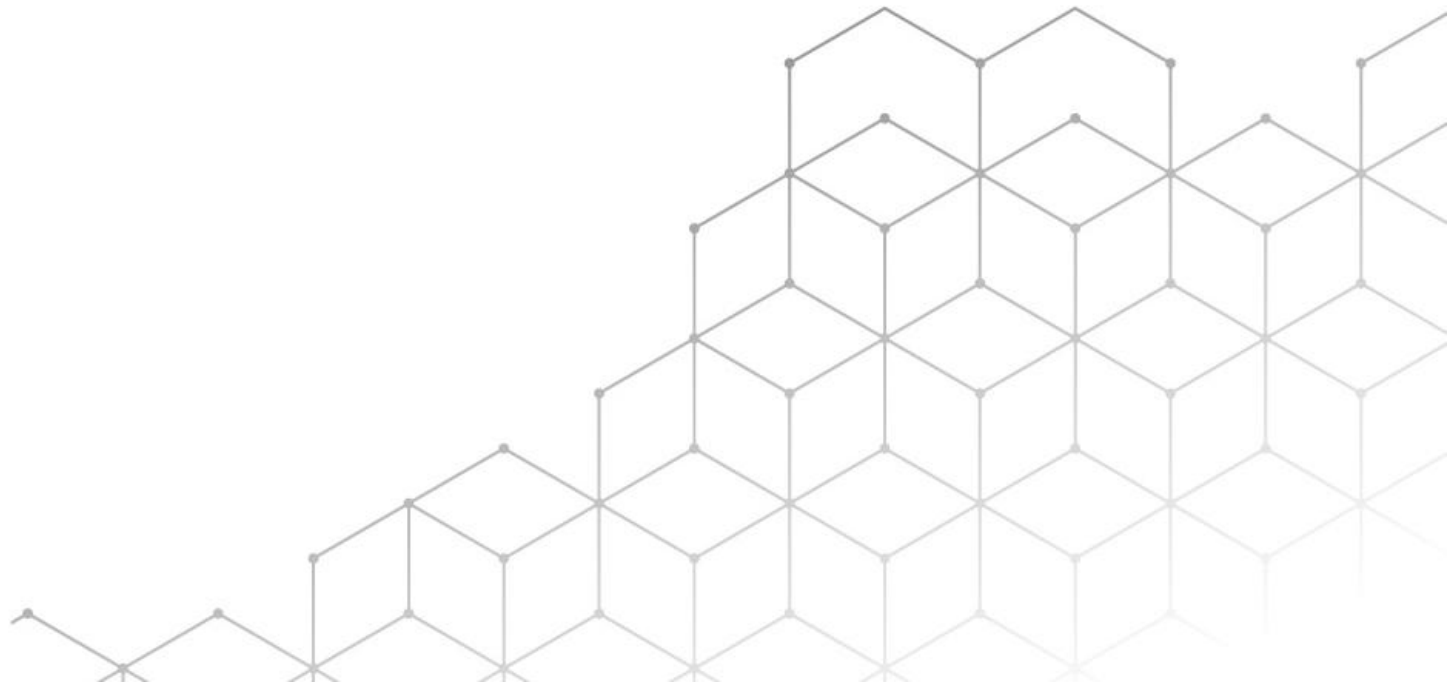


Agenda for Today...

- Types of Data
- Distributed File System (DFS)
- How DFS Works?
- Operations on DFS
- Fault Tolerance by Replication
- Concurrency and Multi-User Access
- Hadoop Distributed File System (HDFS)
- Databricks File System (DBFS)

Multi-user Access

- Multiple users can **read the same file simultaneously** from different nodes.
- Write operations are often **append-only** in DFS to prevent data corruption and conflicts.

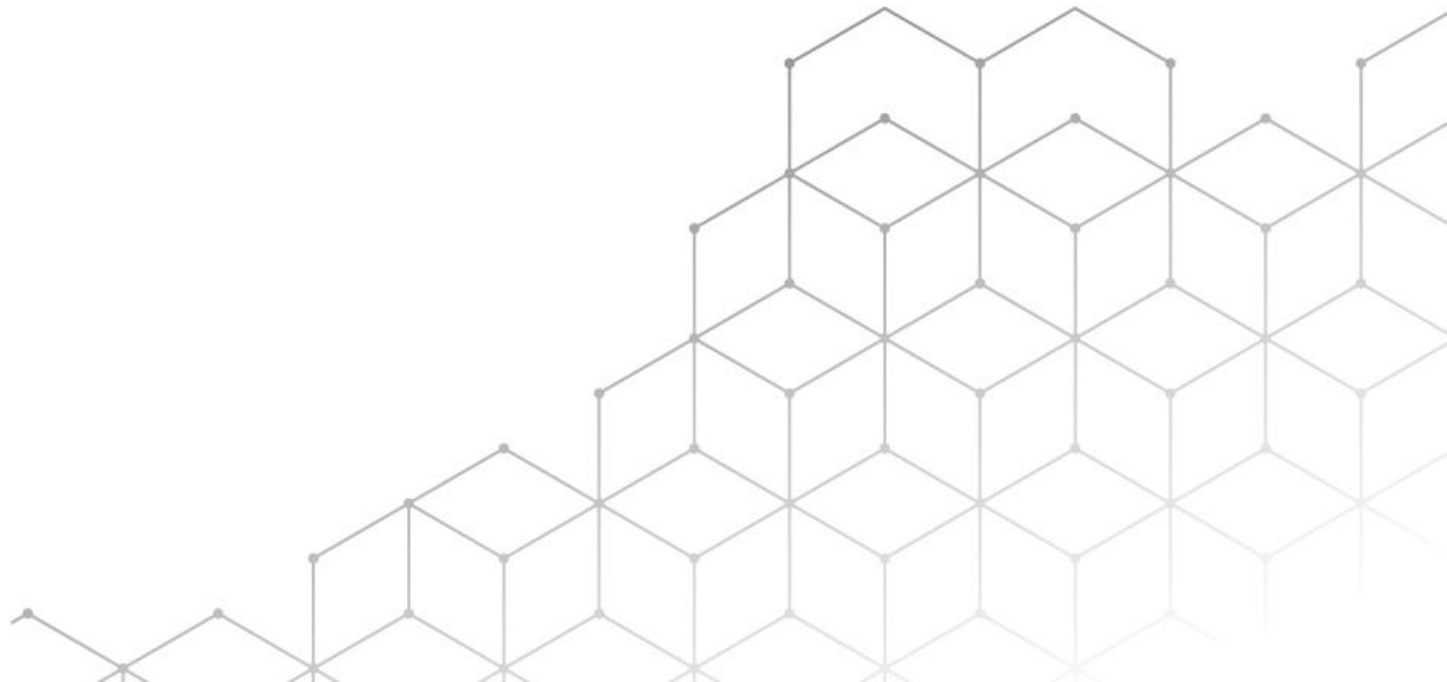


Agenda for Today...

- Types of Data
- Distributed File System (DFS)
- How DFS Works?
- Operations on DFS
- Fault Tolerance by Replication
- Concurrency and Multi-User Access
- Hadoop Distributed File System (HDFS)
 - Master Slave Architecture
 - Data Storage Workflow
 - Replication
 - Rack Awareness
- Databricks File System (DBFS)

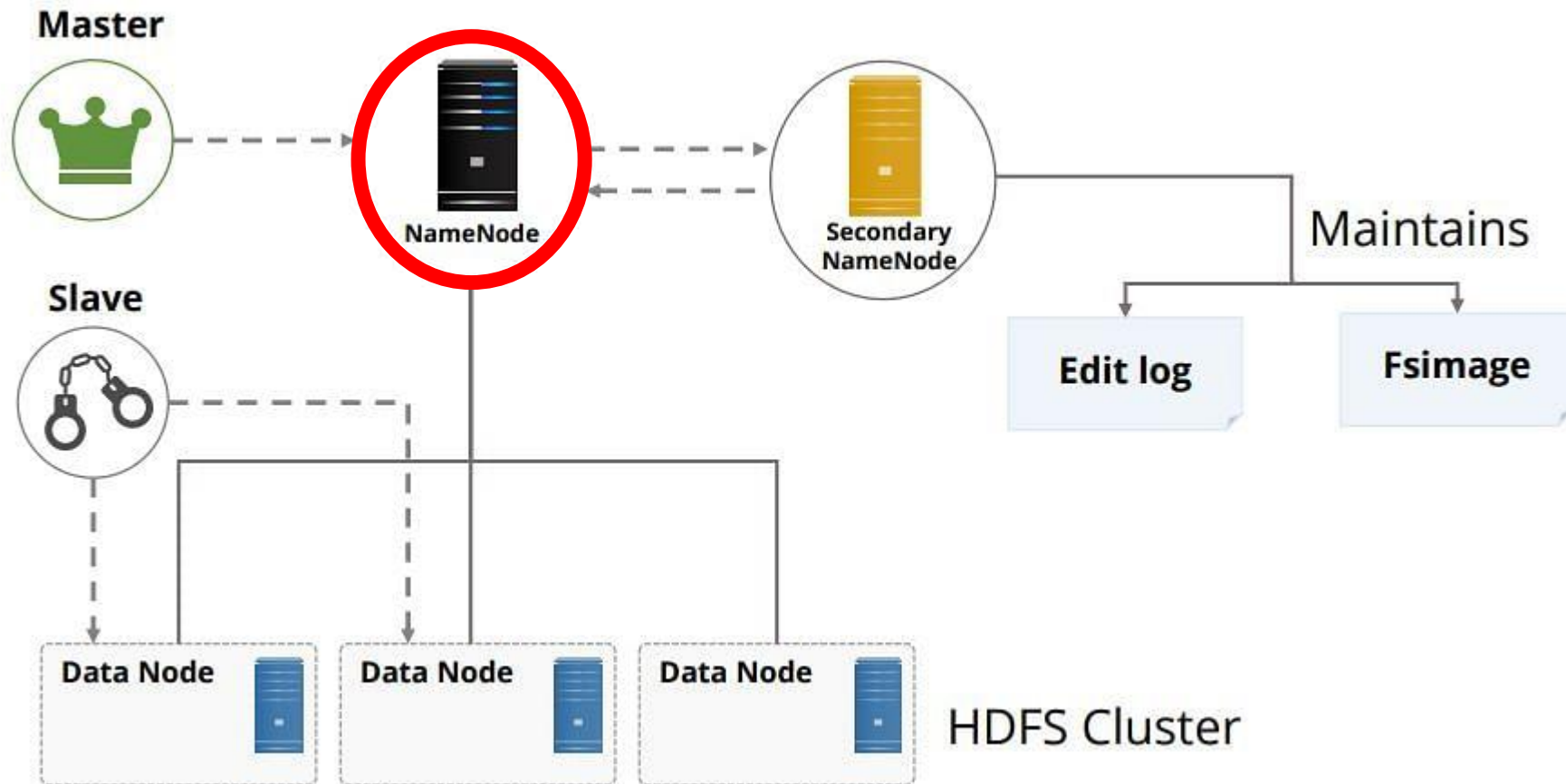
HDFS

- HDFS is a **distributed file system** designed for big data storage.
- Developed as part of the **Hadoop ecosystem**.
- Inspired by **Google File System (GFS)**.



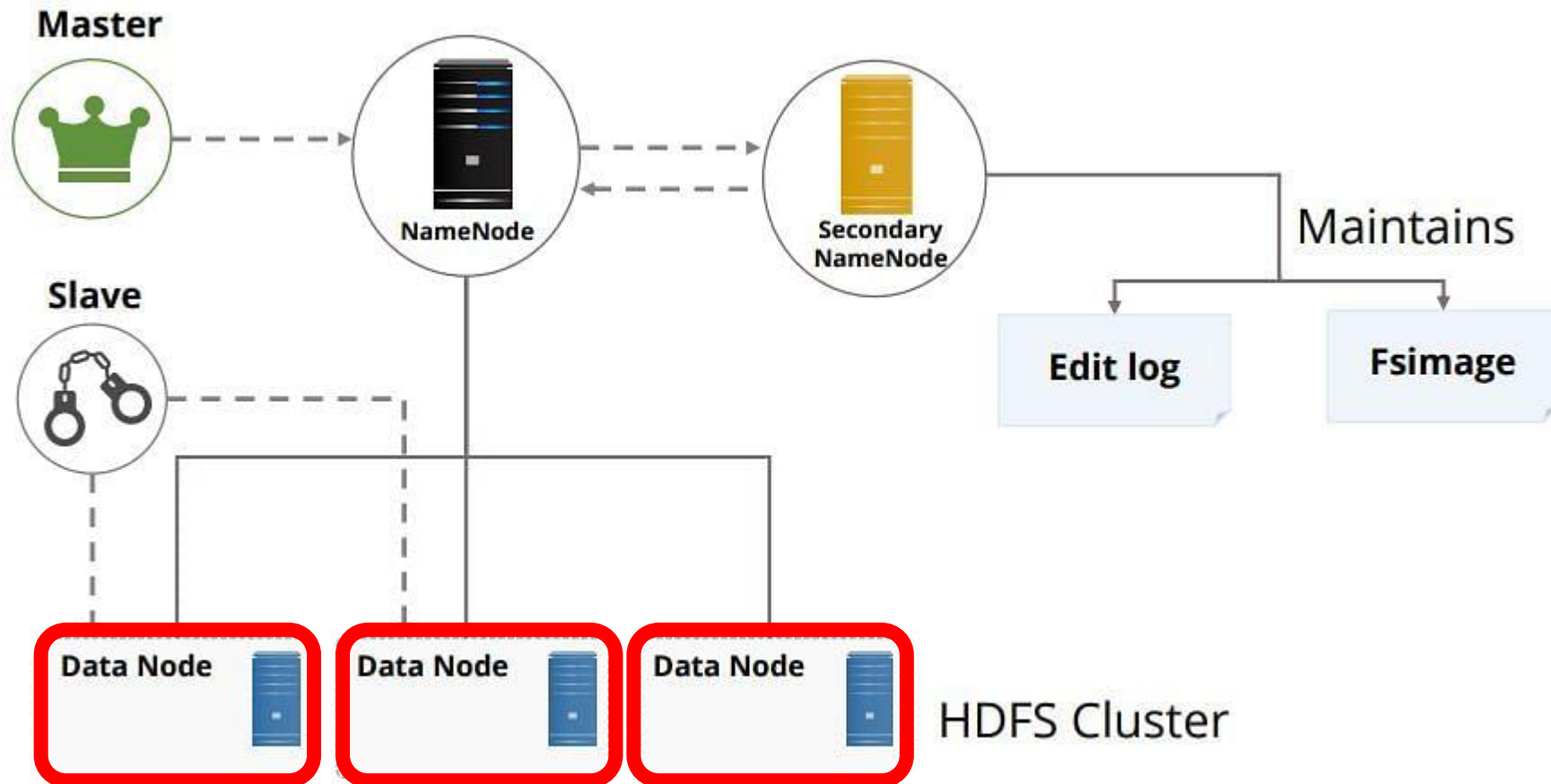
HDFS Master-Slave Architecture

NameNode (Master): Manages metadata and file system namespace.



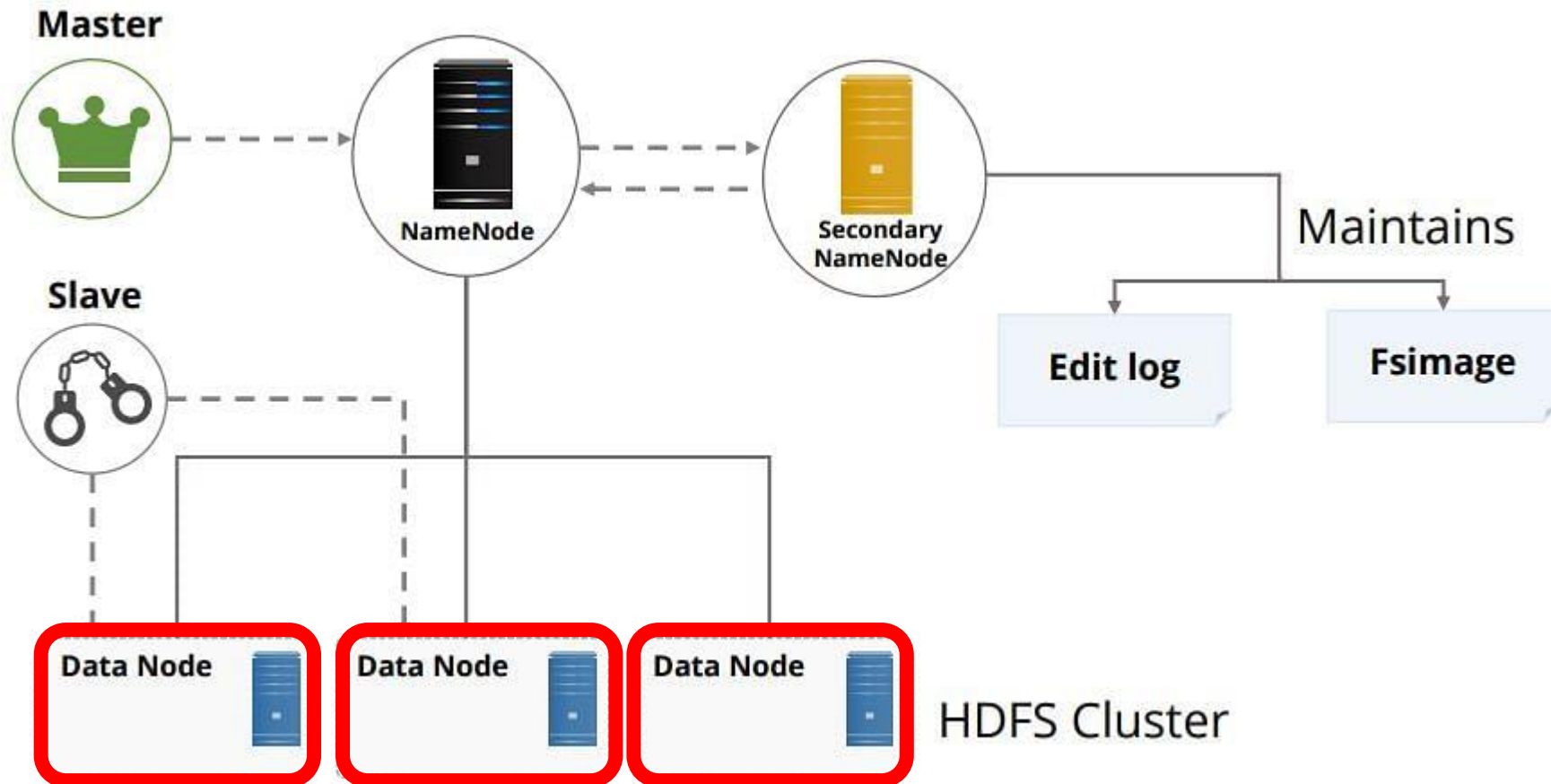
HDFS Master-Slave Architecture

DataNodes (Slaves): Store and manage actual data blocks.



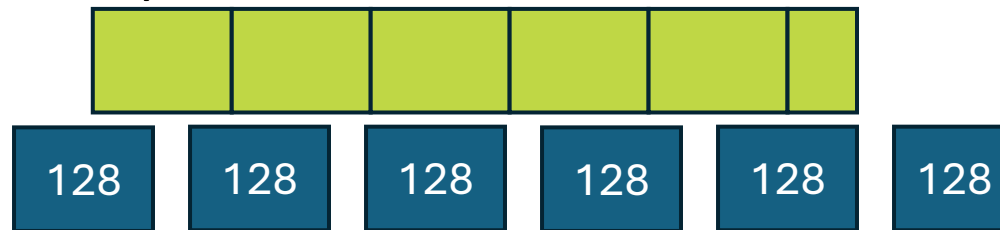
HDFS Master-Slave Architecture

Secondary NameNode: Assists (not a backup NameNode).



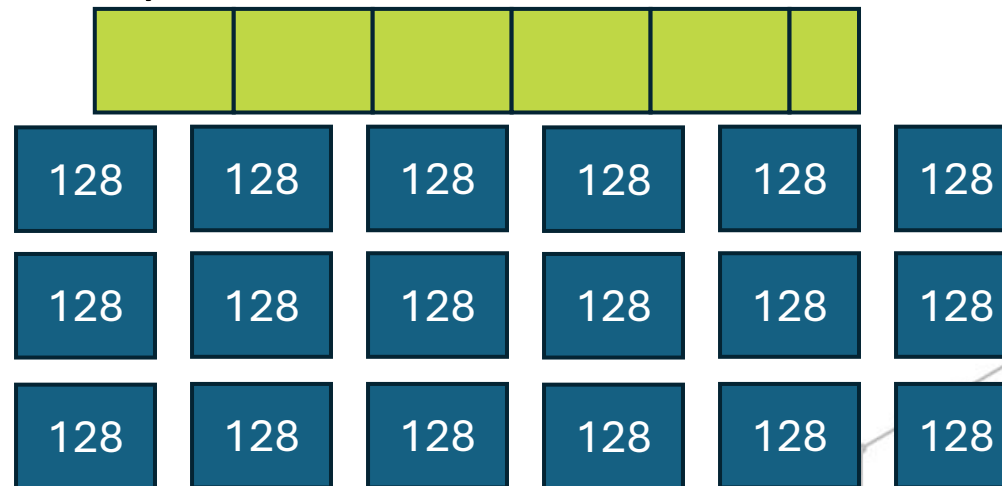
HDFS Data Storage Workflow

1. A file is **split into blocks** (default: 128MB or 256MB).
2. Blocks are **distributed across multiple DataNodes**.
3. The **NameNode tracks block locations** but **does not** store data.
4. **Replication**: Default replication factor is **3** (blocks are stored on three different nodes).



HDFS Data Storage Workflow

1. A file is **split into blocks** (default: 128MB or 256MB).
2. Blocks are **distributed across multiple DataNodes**.
3. The **NameNode tracks block locations** but **does not** store data.
4. **Replication**: Default replication factor is **3** (blocks are stored on three different nodes).



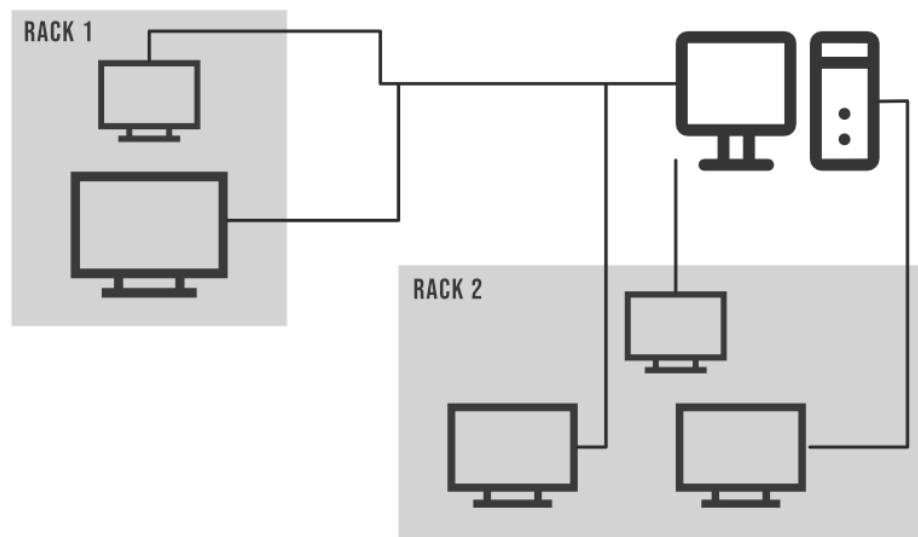
Why Replication?

- **Replication ensures high availability.**
- If a DataNode fails:
 - NameNode detects failure.
 - Replicated blocks are used for recovery.
 - New replicas are created automatically.
- DataNodes send **heartbeat signals** to NameNode to confirm health.



HDFS Rack Awareness

- **Rack-awareness policy:** Spreads data across racks for fault tolerance.
- Ensures replicas are **not stored on the same rack**.
- Improves **data availability and network efficiency**.

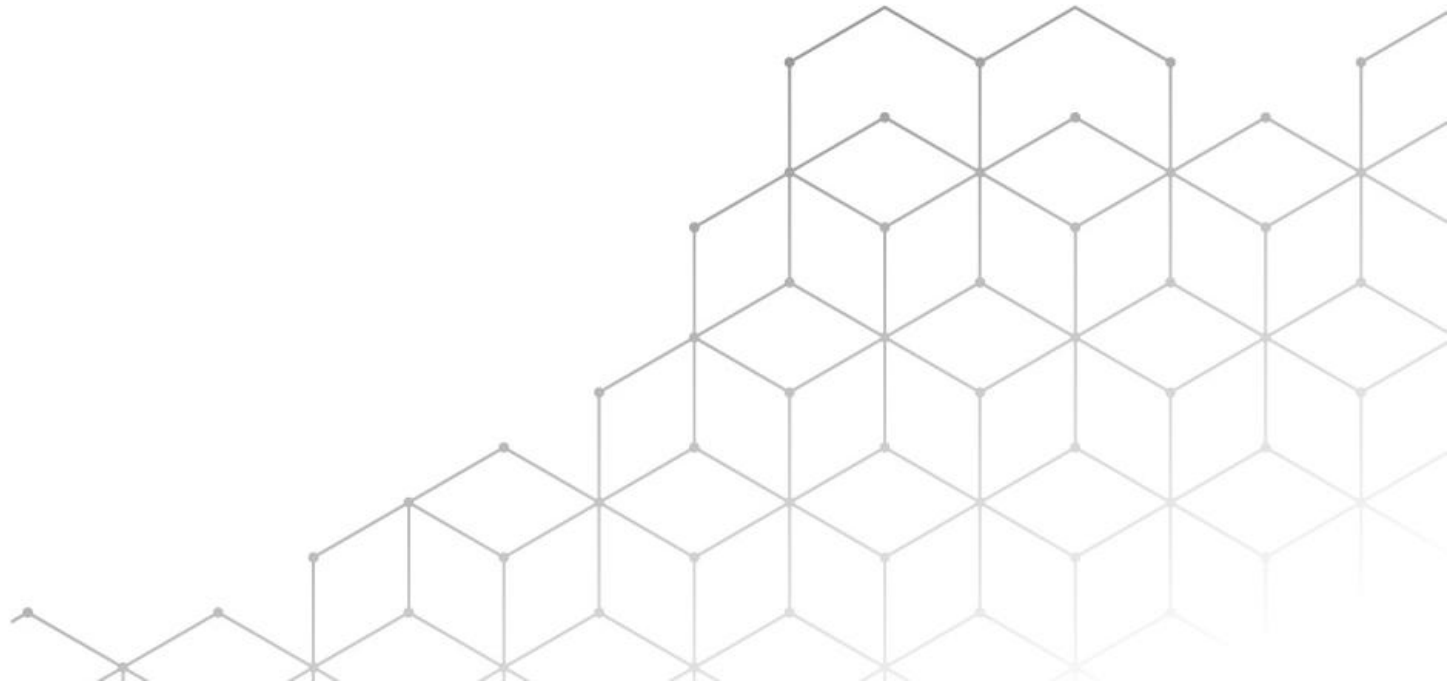


Agenda for Today...

- Types of Data
- Distributed File System (DFS)
- How DFS Works?
- Operations on DFS
- Fault Tolerance by Replication
- Concurrency and Multi-User Access
- Hadoop Distributed File System (HDFS)
- Databricks File System (DBFS)

Databricks File System

- The **Databricks File System (DBFS)** is a distributed **layer of abstraction over cloud object storage** that allows seamless data access and management in **Databricks** environments.



Storage Structure

- DBFS has a hierarchical directory structure similar to local file systems.
- `./databricks/driver` → Stores temporary files on the driver node.
- `./databricks/init` → Stores cluster initialization scripts.
- `./FileStore/` → Stores uploaded files, notebooks, ML models
- `./mnt/` → Used for mounting external cloud storage (S3, ADLS, GCS).

Mounting a File System

Mounting Cloud Storage (AWS S3, Azure Blob, Google Cloud Storage)

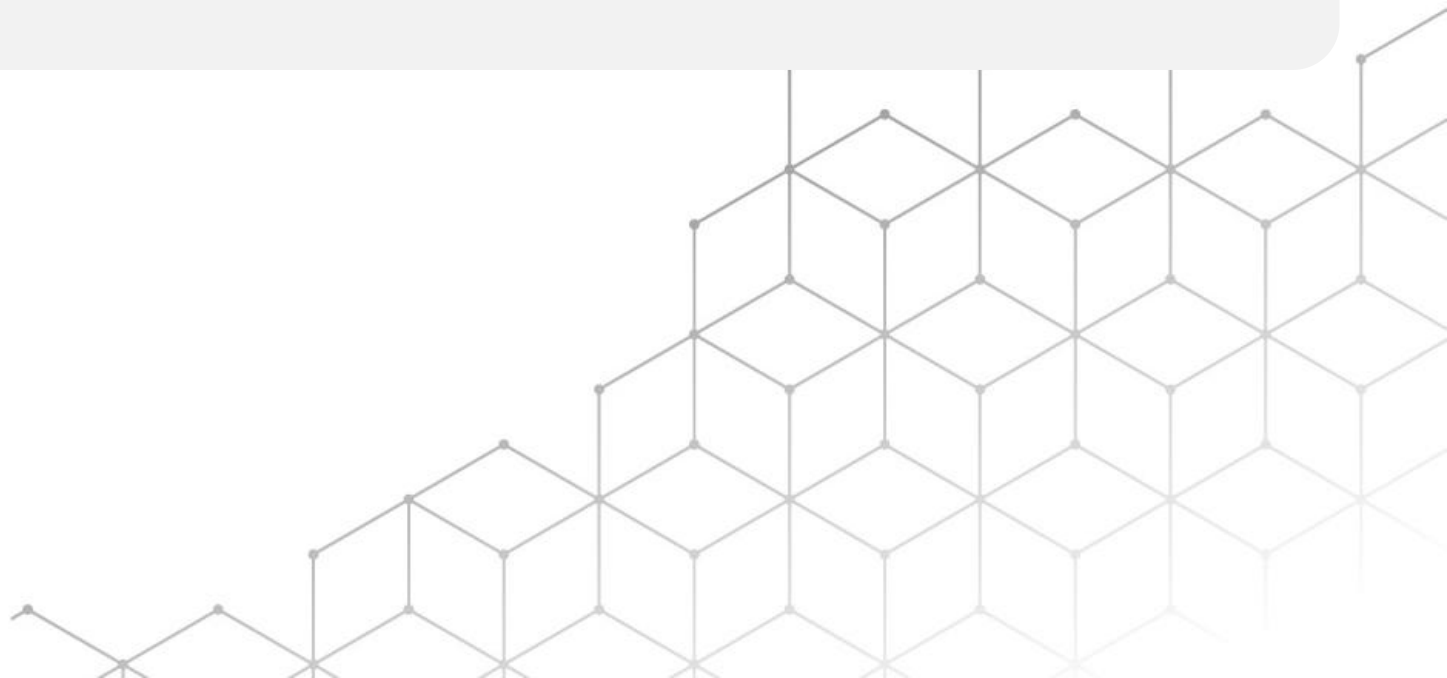
```
dbutils.fs.mount( source="s3a://your-bucket-name", mount_point="/mnt/mydata",  
extra_configs={"fs.s3a.access.key": "your-access-key", "fs.s3a.secret.key": "your-  
secret-key"} )
```

Mounting cloud storage is optional in Databricks, but it depends on how you want to access your data. Databricks provides multiple ways to interact with cloud storage, and mounting is just one of them.

Reading Files

Accessing Files via Spark API

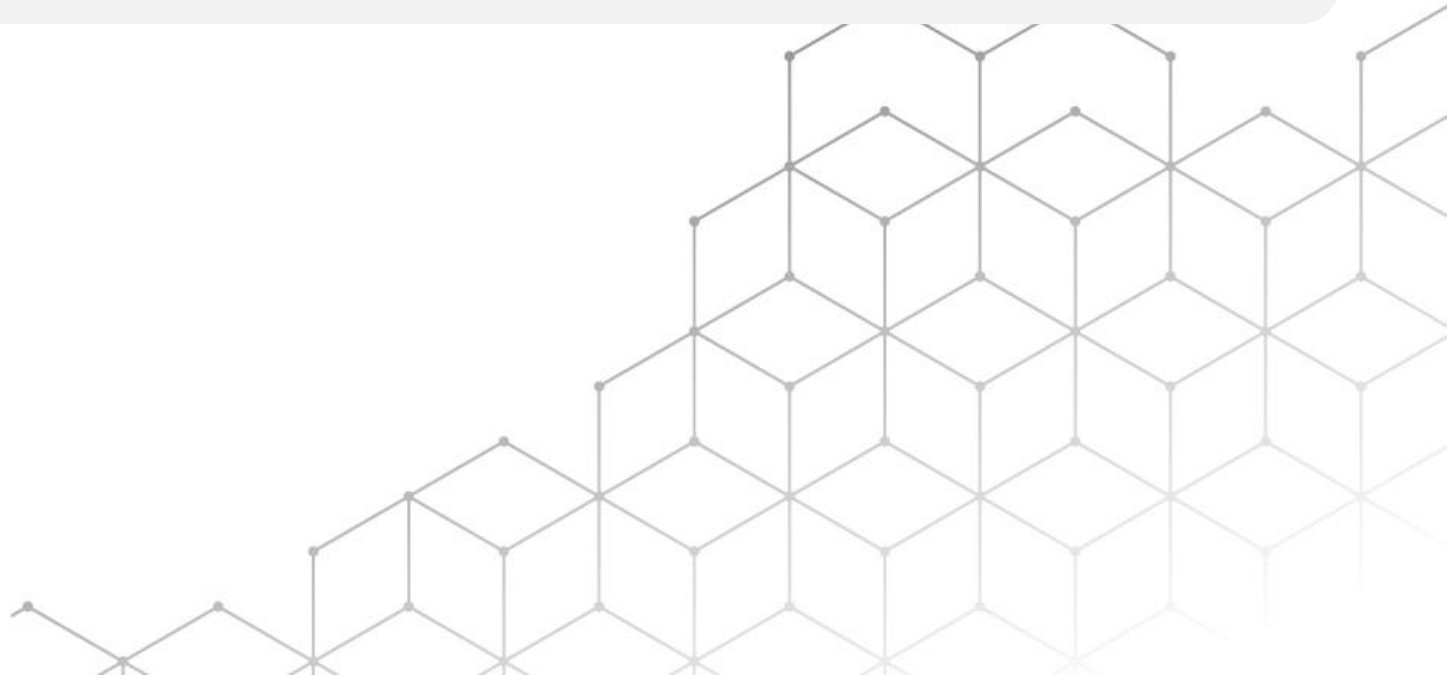
```
df = spark.read.csv("dbfs:/mnt/mydata/sample.csv",  
header=True)  
df.show()
```



Writing Files

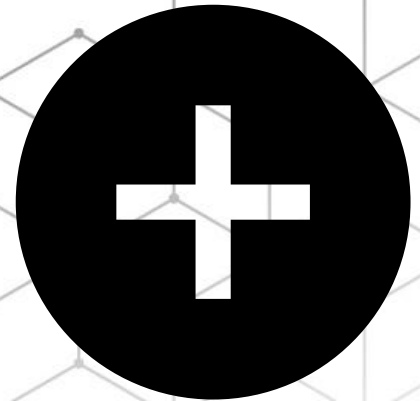
Writing Files to the distributed file system via Spark API

```
df.write.mode("overwrite").parquet("dbfs:/mnt/mydata/output/")
```



Advantages

- Seamless cloud integration (AWS, Azure, GCP).
- Distributed & scalable storage for big data processing.
- Optimized for Apache Spark & ML workflows.



Limited Storage for DBFS Root (dbfs:/)

- The DBFS root (dbfs:/) is ephemeral (temporary) and is not meant for long-term storage.
- Data stored in the root directory is lost when a cluster is restarted or terminated.
- Solution: Use mounts (/mnt/) or direct cloud storage access instead.



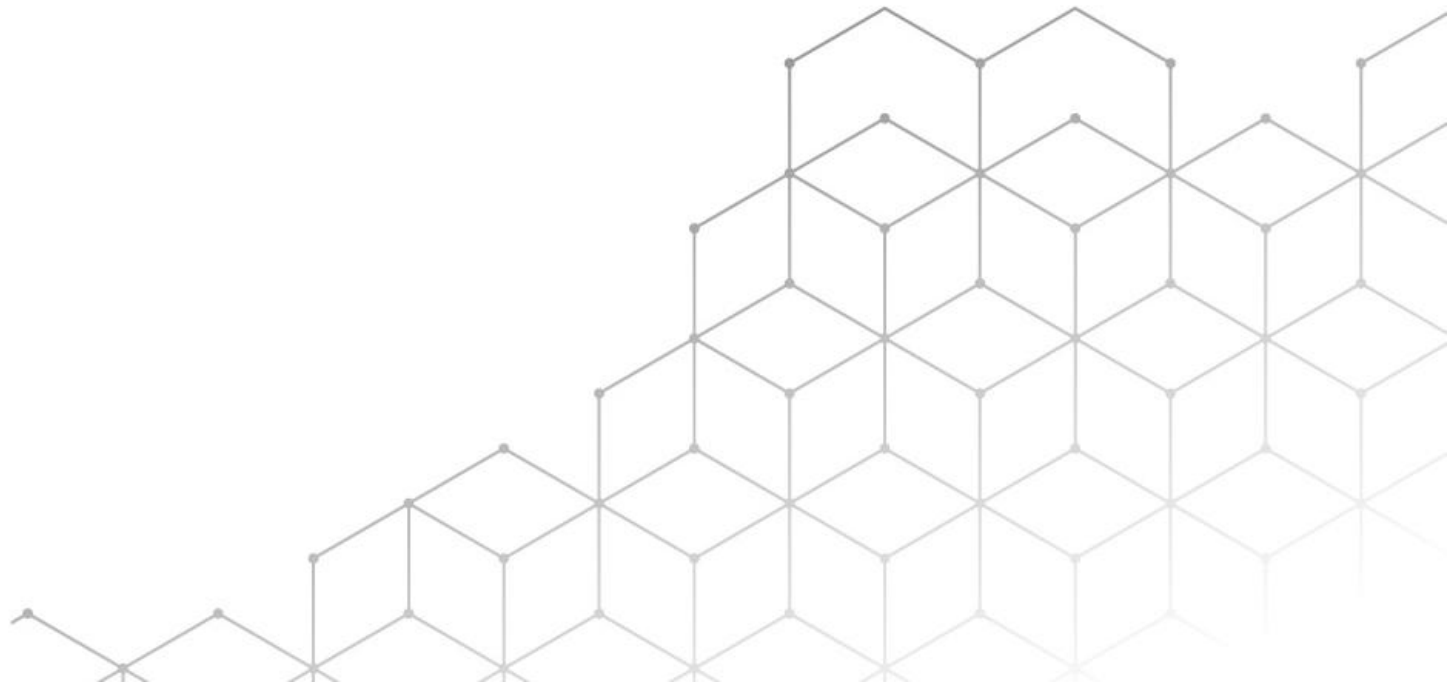
Performance Limitations

- **DBFS adds an extra layer** between Databricks and the underlying cloud storage.
- Large-scale workloads may experience **latency issues** compared to directly using cloud-native storage APIs.
- **Solution:** For high-performance workloads, **use native cloud storage APIs** (S3, ADLS, GCS).



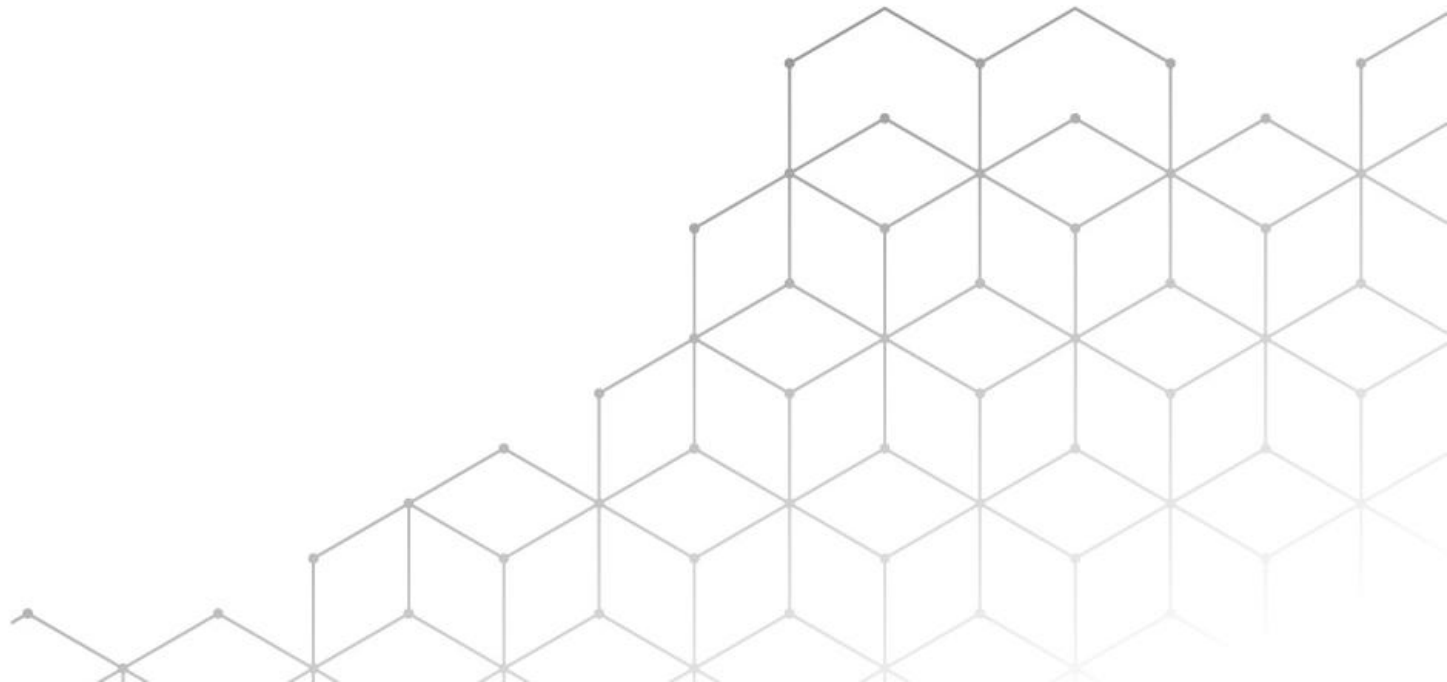
Not Suitable for Real-Time Processing

- **DBFS is optimized for batch processing** but not for **low-latency, real-time streaming** workloads.
- **Solution: Use Apache Kafka, Delta Lake, or cloud-native real-time storage** for streaming data.



Lack of Direct File Locking

- **DBFS does not support traditional file locking**, which can lead to **concurrency issues** if multiple users write to the same file.
- **Solution:** Use **Delta Lake** (which supports ACID transactions) instead of raw DBFS storage for concurrent workloads.



Takeaways

- DBFS is great for simplifying access to cloud storage
- But has security and performance limitations.
- Direct cloud storage access is recommended instead of mounts.
- For transactional and parallel workloads, use Delta Lake instead of raw DBFS storage.



Next Lecture

- Big Data Platforms III:
 - YARN
 - SPARK
 - Databricks



Questions?

The End

